

Laboratórios de Análise de Discurso Político

Francisco Brandão

Lucas M Pavelski

2026-06-03

Table of contents

Introdução	5
Objetivos do curso	6
Motivação	6
Bibliografia	6
Programa detalhado	7
Aula 1 - 06/03/2026:	7
Aula 2 - 20/03/2026:	7
Aula 3 - 10/04/2026:	7
Aula 4 - 15/05/2026:	8
Aula 5 - 24/04/2026:	8
Aula 6 - 22/05/2026:	8
Aula 7 - 29/05/2026:	8
Aula 8 - 26/06/2026:	8
Avaliação	8
1 Linguagem R	9
1.1 Programando scripts em R:	9
1.2 Ambientes para programar em R	10
1.2.1 Rstudio	11
1.2.2 Google Colab	11
1.2.3 Console R	11
1.2.4 Scripts R	11
1.2.5 Notebooks R	12
1.3 Comandos básicos	13
1.3.1 Operadores aritméticos	13
1.3.2 Operadores relacionais e lógicos	13
1.3.3 Variáveis	15
1.3.4 Condicionais	18
1.3.5 Funções	19
1.3.6 Vetores	22
1.3.7 Data frames	25
1.3.8 Bibliotecas e pacotes	27
1.3.9 Strings (cadeias de caracteres)	29
1.4 Exemplo: Análise de Sentimentos com R	30
1.5 Para saber mais	33

1.6	Atividade prática	33
2	Palavras, morfemas e tokens	34
2.1	Obtendo dados textuais	34
2.1.1	Tipos de fontes de dados textuais	35
2.1.2	Exemplo: Carregando Discursos da Câmara	36
2.1.3	Criando o corpus	37
2.2	Níveis de análise textual	41
2.3	A estrutura das palavras: morfemas, lemas e radicais	42
2.3.1	Lematização e Anotação Morfossintática	42
2.3.2	Stemização (Corte do radical)	42
2.3.3	Caracteres e Codificação	43
2.4	Tokenização (Dividindo o texto)	43
2.4.1	A Matriz de Documentos e Termos (DFM)	44
2.5	Stopwords	45
2.6	Palavras no contexto (Keywords-in-context)	45
2.7	Exemplo Prático: Nuvem de Palavras por Partido	47
2.8	Para saber mais	48
2.9	Atividade prática	48
	Glossário	49
	Introdução	49
	Aula 1	49
	Referências	50
	Appendices	51
A	Coletando Dados do legislativo	51
A.1	Coletando dados da API da Câmara dos Deputados	51
A.2	Coletando dados da API do Senado	72

Introdução

Câmara dos Deputados
Centro de Formação, Treinamento e Aperfeiçoamento
Programa de Pós-Graduação



MESTRADO PROFISSIONAL EM PODER LEGISLATIVO PLANO DE CURSO DE DISCIPLINA

DISCIPLINA: ANÁLISE DO DISCURSO POLÍTICO

Período: 1º semestre 2026

Carga horária total: 30 h/a

Código: MEST.7.09.17

PROFESSORES

E-mail

FRANCISCO BRANDÃO, Dr.

francisco.brandao@camara.leg.br

LUCAS MARCONDES PAVELSKI, Dr.

lucas.pavelski@camara.leg.br

CURRÍCULOS RESUMIDOS

FRANCISCO BRANDÃO, Dr.

Grupo de Pesquisa e Extensão (GPE): Ciência de Dados Aplicada ao Estudo do Poder Legislativo: abordagem computacional e métodos de análise

Doutor em Ciência Política pela Universidade da Califórnia, Santa Barbara (2018), nos campos de Política Comparada, Comunicação Política e Tecnologia da Informação e Sociedade. Possui graduação em Jornalismo pela Universidade de São Paulo (2000) e mestrado em Ciência Política pela Universidade de Brasília (2008). Entre 2022 e 2023, foi pesquisador associado da Universidade de Loughborough, no Reino Unido, no projeto PANCOPOP – Comunicação Populista em Tempos de Pandemia. Jornalista na Diretoria de Comunicação da Câmara dos Deputados.

Currículo Lattes: <http://lattes.cnpq.br/9403525530814720>

LUCAS MARCONDES PAVELSKI, Dr.

Doutor e Mestre em Engenharia Elétrica e Informática Industrial pela Universidade Tecnológica Federal do Paraná (2021) com ênfase em sistemas inteligentes. Possui graduação em Ciência da Computação pela Universidade Estadual do Centro-Oeste do Paraná (2012). Atualmente é Analista de TI na Câmara dos Deputados, atuando na Seção de Soluções para a Área Política, com experiência em Ciência de Dados e Engenharia de Software.

Currículo Lattes: <http://lattes.cnpq.br/9287626771427206>

EMENTA DA DISCIPLINA

História, conceitos importantes, questões teóricas e método da Análise de Discurso (AD). Poder como controle, controle do discurso, estratégias discursivas, discurso e poder. Discurso como produção social. Formações discursivas e formações ideológicas. Relação entre estrutura e agência na produção discursiva. Análise argumentativa; discurso e argumentação. As instâncias midiáticas do discurso político. Relações entre discurso político e discurso midiático. Processos hermenêuticos envolvidos na AD. Narrativas, accounts, justificações, estratégias retóricas e uso de significantes vazios na política e nas instâncias jurídicas e midiáticas.

OBJETIVO GERAL DA DISCIPLINA

O aluno deverá compreender e aplicar empiricamente os conceitos básicos relacionados com as principais teorias de análise do discurso e seu instrumental metodológico.

OBJETIVOS ESPECÍFICOS DA DISCIPLINA

Objetivos do curso

Apresentar um ferramental tecnológico para auxiliar na análise de discursos políticos utilizando técnicas de Processamento de Linguagem Natural (PLN).

Motivação

- O volume de dados textuais disponíveis cresceu exponencialmente com a digitalização da informação e o advento das redes sociais.
- Pesquisa quantitativa complementa a pesquisa qualitativa, permitindo análises em larga escala.
- Ferramentas de PLN permitem extrair insights valiosos de grandes volumes de texto.
- Aplicações práticas em diversas áreas, como ciência política, sociologia, marketing e saúde pública.
- Desenvolvimento de habilidades técnicas em análise de dados textuais, programação e modelagem estatística.
- Foco prático do Mestrado Profissional em Poder Legislativo.

Bibliografia

JURAFSKY, D; MARTIN, J. H. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition, 2025.

Livro-texto com referencial teórico sobre processamento de linguagem natural, linguística computacional e análise de conteúdo (Jurafsky and Martin 2026).

Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>

O'NEILL, L et al. Quantitative discourse analysis at Scale—AI, NLP and the transformer revolution. SoDa Laboratories Working Paper Series, v. 35, n. 11, p. 18-26, 2021.

Artigo apresentando NPL do estado-da-arte aplicada a estudos sociais (O'Neill et al. 2021) .

Disponível em: <https://ideas.repec.org/p/ajr/sodwps/2021-12.html>

SILGE, J.; ROBINSON, D. Text mining with R: a tidy approach. Beijing ; Boston: O'Reilly, 2017.

Livro-texto com referencial teórico e prático sobre mineração de textos com R e biblioteca tidytext (Silge and Robinson 2017).

Disponível em: <https://www.tidytextmining.com/>

WICKHAM, Hadley et al. R for data science. Sebastopol: O'Reilly, 2017.

Livro-texto com referencial prático sobre ciência de dados com R (Wickham 2023).

Disponível em: <https://r4ds.hadley.nz/> (inglês) e <https://pt.r4ds.hadley.nz/> (tradução em português).

CASELI, Helena de Medeiros; NUNES, Maria das Graças Volpe. Processamento de linguagem natural: conceitos, técnicas e aplicações em português. 2024.

Livro-texto com referencial teórico sobre processamento de linguagem natural em português (Caseli and Nunes 2024).

Disponível em: <https://brasileiraspln.com/livro-pln/2a-edicao/>

Programa detalhado

Aula 1 - 06/03/2026:

- Aplicações de PLN em análise de discurso político
- Linguagem R
- Operações básicas com textos em R

Aula 2 - 20/03/2026:

- Obtenção de dados textuais
- Dados do legislativo
- Morfemas, lemas e stemming
- Tokens e stopwords

Aula 3 - 10/04/2026:

- n-gramas
- Collocations
- TF-IDF
- Análise de tópicos

Aula 4 - 15/05/2026:

- Análise sintática
- Embeddings
- Similaridade de Documentos

Aula 5 - 24/04/2026:

- Classificação de textos
- Modelos de linguagem

Aula 6 - 22/05/2026:

- Análise de sentimentos
- Reconhecimento de entidades nomeadas

Aula 7 - 29/05/2026:

- Modelos discursivos
- Retrieval Augmented Generation (RAG)

Aula 8 - 26/06/2026:

- Apresentações finais

Avaliação

A avaliação será feita com base na participação nas aulas e na apresentação final de um projeto prático de análise de discurso político utilizando técnicas de processamento de linguagem natural.

1 Linguagem R

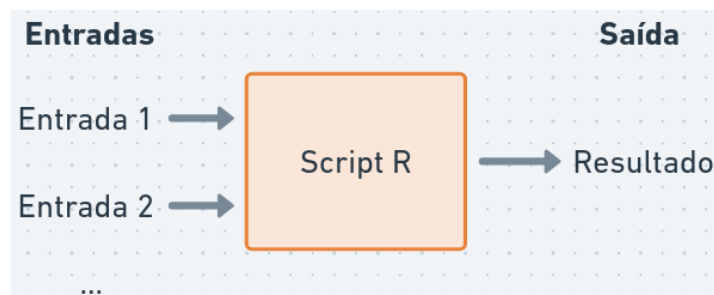
i Nesta aula

- Como utilizar a linguagem R
- Comandos básicos
- Utilizando bibliotecas
- Funções para processamento de texto

Link para Google Colab: <https://colab.research.google.com/drive/1LalxshzmyMmTyfuIaFbp7RYcoP5Qnvuo>

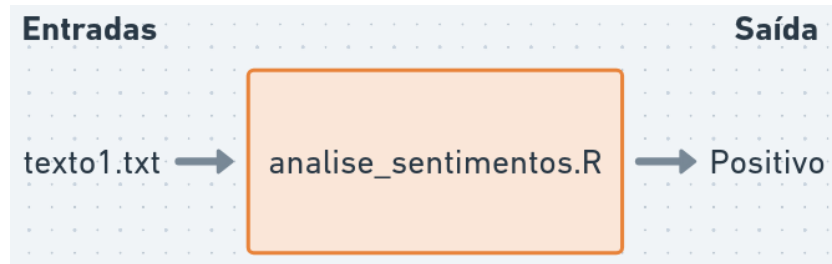
- **R** é uma linguagem de programação e ambiente de software para computação estatística e gráficos
- Criada por Ross Ihaka e Robert Gentleman em 1993
- Muito utilizada em ciência de dados, estatística e análise de texto
- Possui uma grande comunidade de usuários e desenvolvedores, com muitas bibliotecas para análise de textos

1.1 Programando scripts em R:



Script R: sequência de comandos R que podem ser executados para realizar uma tarefa específica (ex: análise de sentimentos)

Em PLN:



`analise_sentimentos.R` é um arquivo contendo instruções para:

- Ler arquivos de texto
- Processar textos (tokenização, remoção de stopwords, stemming)
- Analisar sentimentos (encontrar palavras positivas e negativas)
- Gerar saída (relatório, gráfico, tabela)

O **interpretador R (Rscript)** é um programa que executa os comandos do script e produz a saída desejada. Por exemplo, pelo terminal de comandos:

```
{ $ Rscript analise_sentimentos.R texto1.txt } > "Positivo"
```

1.2 Ambientes para programar em R

O interpretador R está disponível para download em <https://cran-r.c3sl.ufpr.br/>

```
## (código utilizado apenas para exibir páginas no Google Colab)
if (!requireNamespace("htmltools", quietly = TRUE)) {
  install.packages("htmltools")
}
htmltools::tags$iframe(src = "https://cran-r.c3sl.ufpr.br/", width = "100%", height = "315")
```

- No Windows, é necessário também instalar o RTools, disponível em <https://cran.r-project.org/bin/windows/Rtools/>, que inclui ferramentas para auxiliar a instalação de pacotes do R.

Podemos escrever comandos R em diversos ambientes:

1.2.1 Rstudio

- Ambiente de programação integrado (IDE) instalada localmente (Windows, Mac, Linux).
- Possui diversas funcionalidades para facilitar a escrita e execução de código R, como editor de scripts, console interativo, visualização de gráficos e gerenciamento de pacotes.

Disponível em: <https://posit.co/downloads/>.

Uma alternativa é o IDE Positron, disponível em: <https://positron.posit.co/download.html>.

1.2.2 Google Colab

- Ambiente online para programação em Python, mas também suporta R com algumas configurações adicionais (selecione “R” como linguagem de programação para criar um notebook R).
- Permite compartilhar notebooks interativos com código R e resultados.

Disponível em: <https://colab.research.google.com/>.

Outra alternativa é a posit.cloud, disponível em: <https://posit.cloud>.

1.2.3 Console R

- Interface de linha de comando para executar comandos R diretamente no terminal.
- Um comando é executado por vez (digite o comando e aperte Enter), e o resultado é exibido imediatamente.
- É um ambiente menos amigável para desenvolver scripts longos, mas útil para testes rápidos e para o aprendizado da linguagem.

Acessando o console R:

- No terminal, digite R para iniciar o console R e q() para sair.
- No RStudio, o console R está disponível na parte inferior da interface.
- No Google Colab, clique em “Terminal” e digite “R” para iniciar o console R.

1.2.4 Scripts R

- Arquivo de texto com extensão .R que contém uma sequência de comandos R.
- Permite organizar e reutilizar código, facilitando a execução de tarefas complexas.

Como escrever um script R:

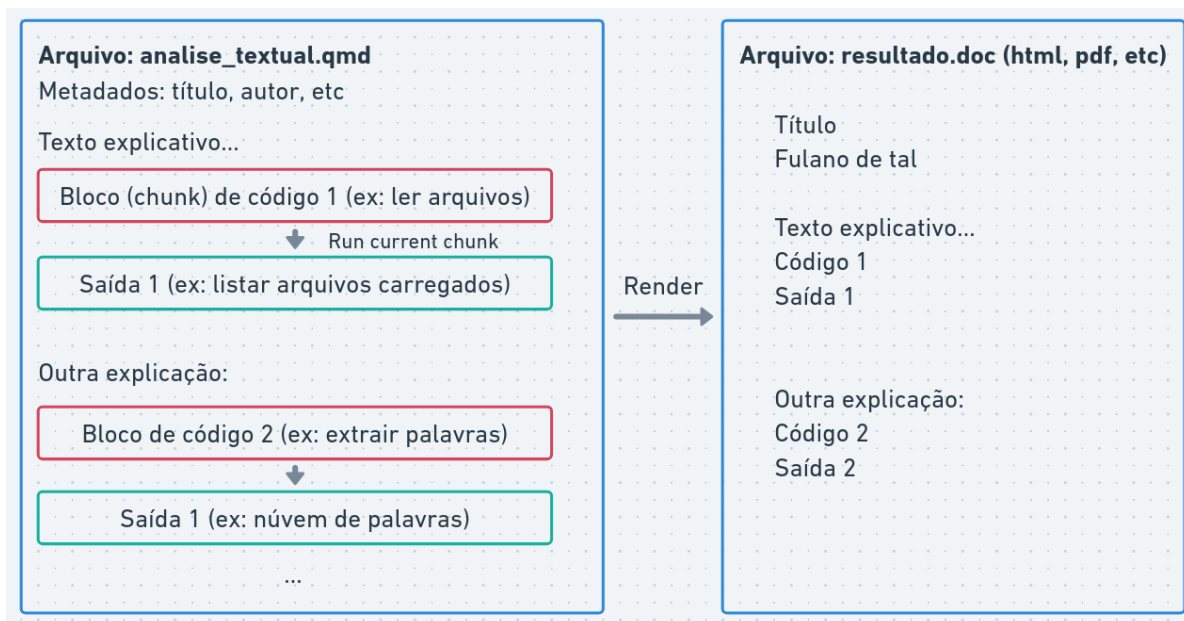
- No RStudio, crie um novo script em: **File > New File > R Script**. Clique em “Source” para executar o script inteiro ou selecione linhas específicas e clique em “Run” para executar apenas essas linhas.

1.2.5 Notebooks R

- Notebooks são documentos que combinam texto explicativo, código R e resultados (gráficos, tabelas) em um formato interativo.
- Permitem criar relatórios dinâmicos e interativos, facilitando a comunicação dos resultados.

Para criar e executar notebooks R, você pode usar o Rstudio ou o Google Colab:

- No Rstudio, crie um novo notebook: **File > New File > Quarto Document**.
- No Google Colab, clique em “File” > “New Notebook” e selecione “R” como linguagem de programação.



Notebooks possuem dois tipos de células:

- células de texto para escrever explicações e comentários
- células de código para escrever e executar códigos em R (o botão “play” executa cada célula individualmente).

1.3 Comandos básicos

- Executar cálculos
- Operadores relacionais e lógicos
- Variáveis
- Listas e vetores
- Funções
- Strings (cadeias de caracteres)

1.3.1 Operadores aritméticos

R pode ser utilizado para cálculos básicos, por exemplo:

```
print(4 + 3)
```

```
[1] 7
```

- `print(*)` é uma função que exibe o resultado de uma expressão na tela.

```
print(5 * 2 + 3)
```

```
[1] 13
```

```
print((6 - 1) / 2)
```

```
[1] 2.5
```

Qualquer código após `#` é considerado um comentário e não é executado:

```
print(8 / 3)      # Isto é um comentário, não será executado
```

```
[1] 2.666667
```

1.3.2 Operadores relacionais e lógicos

Operadores relacionais são utilizados para comparar valores.

Igualdade:

```
print(5 == 5)
```

[1] TRUE

Diferença:

```
print(1 != 2)
```

[1] TRUE

Maior que:

```
print(4 > 3)
```

[1] TRUE

Menor que:

```
print(2 < 5)
```

[1] TRUE

Maior ou igual a:

```
print(3 >= 3)
```

[1] TRUE

- Expressões lógicas: retornam verdadeiro (TRUE) ou falso (FALSE)

Conjunção “e”:

```
print(4 > 3 && 8 > 2)
```

[1] TRUE

- `&&` é o operador lógico “e” (conjunção) que retorna TRUE se ambas as expressões forem verdadeiras, caso contrário retorna FALSE:

- `TRUE && TRUE` retorna `TRUE`
- `TRUE && FALSE` retorna `FALSE`
- `FALSE && TRUE` retorna `FALSE`
- `FALSE && FALSE` retorna `FALSE`

Disjunção “ou”:

```
print(4 > 3 || 2 > 8)
```

[1] `TRUE`

- `||` é o operador lógico “ou” (disjunção), que retorna `TRUE` se pelo menos uma das expressões for verdadeira. Caso contrário, retorna `FALSE`.
- `TRUE || TRUE` retorna `TRUE`
- `TRUE || FALSE` retorna `TRUE`
- `FALSE || TRUE` retorna `TRUE`
- `FALSE || FALSE` retorna `FALSE`

Negação “não”

```
print(!(5 > 3))
```

[1] `FALSE`

- `!` é o operador lógico “não” (negação) que inverte o valor lógico de uma expressão. Se a expressão for `TRUE`, retorna `FALSE`, e se for `FALSE`, retorna `TRUE`.
- `!(TRUE)` retorna `FALSE`
- `!(FALSE)` retorna `TRUE`

```
print((TRUE && FALSE) || TRUE)
```

[1] `TRUE`

1.3.3 Variáveis

Variáveis são utilizadas para guardar valores na memória do computador:

```
a <- 5  
  
print(a)
```

[1] 5

Utilizamos o operador <- para atribuir valores a variáveis.

O valor armazenado pode ser utilizado em outras expressões:

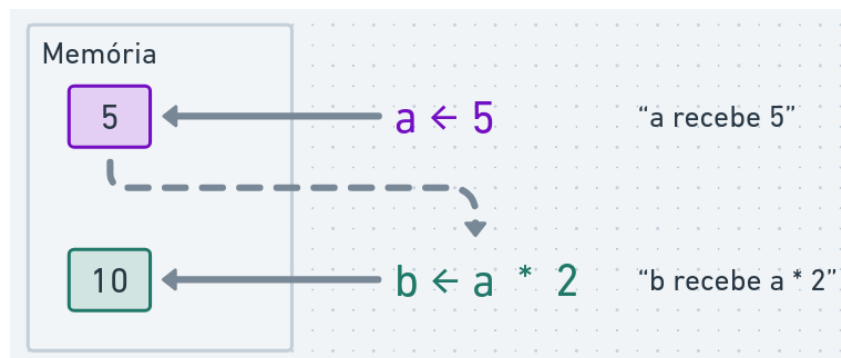
```
print(a * 2 + 3)
```

[1] 13

Ou podemos criar novas variáveis a partir de outras:

```
b <- a * 2  
  
print(b)
```

[1] 10



O valor armazenado na variável pode ser alterado:

```
a <- 10  
  
print(a)
```



```
[1] 10
```

Variáveis podem armazenar diferentes tipos:

```
p <- 1 == 1  
q <- 4 < 3  
  
print(p || q)
```

```
[1] TRUE
```

O nome de uma variável deve começar com uma letra e pode conter letras, números (exceto no começo), underscores (_) e pontos:

```
isto_é_uma_variável <- 10  
aluno_aprovado_1 <- isto_é_uma_variável == 10  
  
print(isto_é_uma_variável)
```

```
[1] 10
```

```
print(aluno_aprovado_1)
```

```
[1] TRUE
```

Exemplo de cálculo do índice de massa corporal (IMC):

```
peso <- 70 # peso em kg  
altura <- 1.75 # altura em metros  
  
imc <- peso / (altura * altura)  
  
imc_saudavel <- imc >= 18.5 && imc <= 24  
  
print(imc)
```

```
[1] 22.85714
```

```
print(imc_saudavel)
```

```
[1] TRUE
```

1.3.4 Condicionais

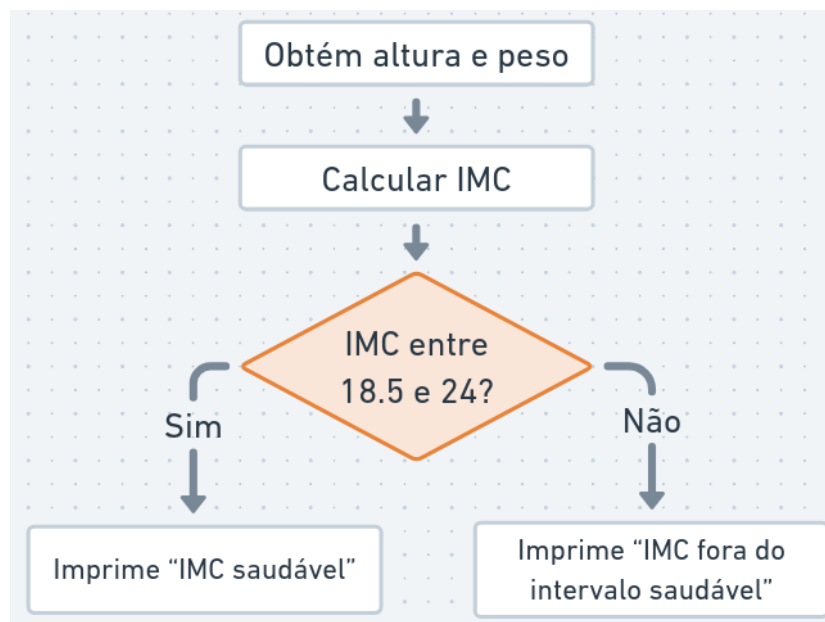
O comando `if` é utilizado para executar um bloco de código apenas se uma condição for verdadeira:

```
x <- 10

if (x > 5) {
  print("x é maior que 5")
}
```

```
[1] "x é maior que 5"
```

O comando `else` pode ser utilizado para executar um bloco de código alternativo caso a condição seja falsa:



```
peso <- 70 # peso em kg
altura <- 1.75 # altura em metros

imc <- peso / (altura * altura)

print(imc)
```

```
[1] 22.85714
```

```
imc_saudavel <- imc >= 18.5 && imc <= 24

if (imc_saudavel) {
  print("IMC saudável")
} else {
  print("IMC fora do intervalo saudável")
}
```

```
[1] "IMC saudável"
```

1.3.5 Funções

Uma **função** é um bloco de código que possui um nome e realiza uma tarefa específica, geralmente para operações comuns que se repetem.

Por exemplo, **print** é a função que usamos para exibir resultados na tela.

R possui várias funções para estatística e análise de dados. Alguns exemplos são:

Valor absoluto:

```
print(abs(-5))
```

```
[1] 5
```

Raiz quadrada:

```
raiz <- sqrt(16)

print(raiz)
```

[1] 4

Logaritmo:

```
log(100, base = 10)
```

[1] 2

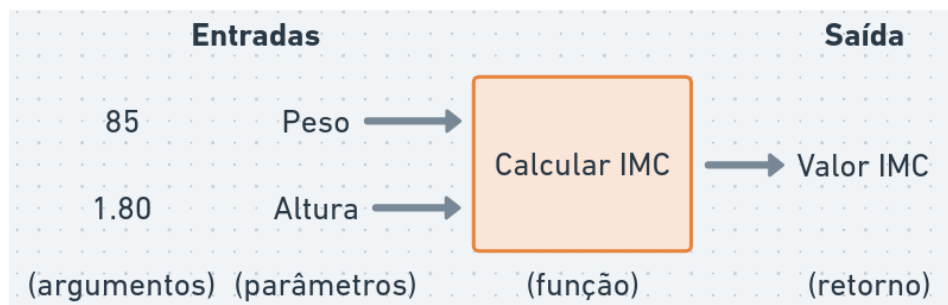
- O número 100 é o primeiro argumento da função.
- O número 10 é o segundo argumento, nomeado como base. Por padrão, a base é o número de Euler ($\exp(1)$), calculando o logaritmo natural.
- Para obter ajuda sobre uma função, execute `help(log)` ou `?log`.

Também é possível definir nossas próprias funções:

```
calcular_imc <- function(peso, altura) {  
  altura_quadrado <- altura * altura  
  return(peso / altura_quadrado)  
}
```

```
imc <- calcular_imc(85, 1.80)  
print(imc)
```

[1] 26.23457



Observe a sintaxe para declarar uma função:

```
nome_da_funcao <- function(argumentos) {  
  # corpo da função  
}
```

A expressão `return(...)` é utilizada para indicar o valor que a função deve retornar. Se `return()` não for especificado, a função retornará o resultado da última expressão avaliada em seu corpo.

Note que qualquer variável criada dentro de uma função é uma variável **local**, ou seja, ela não pode ser acessada fora do escopo da função:

```
print(altura_quadrado)
```

Error in print(altura_quadrado): objeto 'altura_quadrado' não encontrado

Após definir uma função, podemos utilizá-la em qualquer parte do código:

```
teste_imc <- function(peso, altura) {  
  imc <- calcular_imc(peso, altura)  
  imc_saudavel <- imc >= 18.5 && imc <= 24  
  return(imc_saudavel)  
}  
  
print(calcular_imc(80, 1.75) < calcular_imc(70, 1.65))
```

[1] FALSE

Em certos casos, utilizamos o resultado de uma função como argumento de outra função:

```
converter_para_celsius <- function(fahrenheit) {  
  celsius <- (fahrenheit - 32) * 5 / 9  
  return(celsius)  
}  
  
esta_congelando <- function(celsius) {  
  congelando <- celsius <= 0  
  return(congelando)  
}  
  
temperatura_inverno <- 25 # Graus Fahrenheit (aprox. -3.8 °C)  
  
resultado <- esta_congelando(converter_para_celsius(temperatura_inverno))  
  
print(resultado)
```

```
[1] TRUE
```

Em R, podemos utilizar o *operador pipe* (`|>`) para encadear funções de forma mais legível:

```
temperatura_inverno <- 25 # Graus Fahrenheit (aprox. -3.8 °C)

resultado <- temperatura_inverno |>
  converter_para_celsius() |>
  esta_congelando()

print(resultado)
```

```
[1] TRUE
```

- No exemplo, a expressão `temperatura_inverno` é passada como primeiro argumento para a função `converter_para_celsius()`, e o resultado dessa função é então passado como primeiro argumento para a função `esta_congelando()`.

O operador pipe (`|>`) torna o código mais legível, permitindo que as funções sejam encadeadas de maneira clara e fluida.

1.3.6 Vetores

Vetores são estruturas que armazenam uma sequência de elementos do mesmo tipo:

```
vetor_num <- c(1, 2, 3, 4, 5, 6, 7)

print(vetor_num)
```

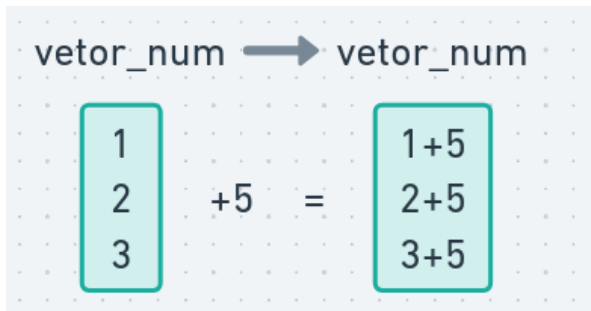
```
[1] 1 2 3 4 5 6 7
```

Operações aritméticas podem ser aplicadas a todos os elementos de um vetor simultaneamente:

```
vetor_num <- c(1, 2, 3)
vetor_num <- vetor_num + 5

print(vetor_num)
```

```
[1] 6 7 8
```



É possível criar sequências numéricas de forma simples:

```
vetor_seq <- 1:10  
  
print(vetor_seq)
```

```
[1] 1 2 3 4 5 6 7 8 9 10
```

Algumas funções operam sobre vetores:

```
print(sum(1:100))
```

```
[1] 5050
```

```
print(mean(1:100))
```

```
[1] 50.5
```

```
print(sd(1:100))
```

```
[1] 29.01149
```

Para acessar um elemento específico do vetor, utilizamos colchetes []:

```
vetor_pares <- c(2, 4, 6, 8, 10)  
  
print(vetor_pares[3])
```

```
[1] 6
```

Para alterar elementos do vetor também utilizamos colchetes:

```
vetor_seq <- 1:10
vetor_seq[3] <- 999

print(vetor_seq)
```

```
[1] 1 2 999 4 5 6 7 8 9 10
```

Podemos acessar múltiplos elementos do vetor:

```
print(vetor_pares[c(1, 4)])
```

```
[1] 2 8
```

Também podemos filtrar elementos do vetor com expressões lógicas:

```
pares_maior_5 <- vetor_pares > 5
elementos_maior_5 <- vetor_pares[pares_maior_5]

print(vetor_pares)
```

```
[1] 2 4 6 8 10
```

```
print(pares_maior_5)
```

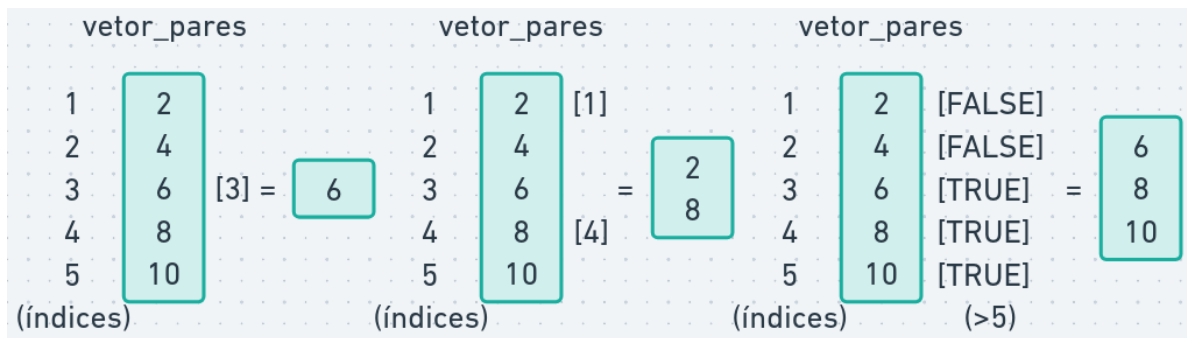
```
[1] FALSE FALSE TRUE TRUE TRUE
```

```
print(elementos_maior_5)
```

```
[1] 6 8 10
```

```
print(vetor_pares[vetor_pares > 5])
```

```
[1] 6 8 10
```

O operador `%in%` é utilizado para verificar se certos elementos estão presentes em um vetor:

```
vetor_a <- c(2, 3, 10)
print(3 %in% vetor_a)
```

```
[1] TRUE
```

```
vetor_b <- c(3, 4, 5, 6, 7)
print(vetor_a %in% vetor_b)
```

```
[1] FALSE TRUE FALSE
```

1.3.7 Data frames

R possui vários tipos de dados, os principais são:

- Numéricos: números inteiros ou decimais (ex: 5, 3.14)
- Lógicos: TRUE ou FALSE
- Strings: texto entre aspas (ex: "Olá, mundo!")

Data frames são estruturas de dados tabulares, onde cada coluna é um vetor e cada linha representa uma observação.

```
df <- data.frame(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  turma = c("A", "B", "A")
)
print(df)
```

	nome	idade	turma
1	João	30	A
2	Maria	25	B
3	Pedro	28	A

Data frames são amplamente utilizados para análise de dados, permitindo manipular e analisar conjuntos de dados tabulares.

Essencialmente, um data frame é uma lista de vetores de mesmo comprimento, em que cada vetor representa uma coluna. Cada coluna pode conter um tipo diferente de dado (numérico, lógico, texto etc.).

Acessando colunas:

```
df <- data.frame(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  turma = c("A", "B", "A")
)

print(df$idade)
```

```
[1] 30 25 28
```

```
print(df[, "idade"])
```

```
[1] 30 25 28
```

Acessando linhas:

```
print(df[1, ])
```

	nome	idade	turma
1	João	30	A

Acessando linha e coluna:

```
print(df[1, "idade"])
```

```
[1] 30
```

Para selecionar alunos com mais de 25 anos:

```
alunos_mais_velhos <- df[df$idade > 25, ]
print(alunos_mais_velhos)
```

```
      nome idade turma
1  João     30      A
3 Pedro     28      A
```

1.3.8 Bibliotecas e pacotes

Bibliotecas (ou pacotes) são conjuntos de funções, dados e documentação que estendem as funcionalidades do R. Geralmente são escritas por outros usuários e compartilhadas com a comunidade (ex: `tidyverse`).

Para instalar uma biblioteca, utilizamos o comando `install.packages()`.

```
if (!"tidyverse" %in% installed.packages()) {
  install.packages("tidyverse")
}
```

Pesquisadores do mundo todo contribuem com bibliotecas no repositório oficial do R, <https://cran-r.c3sl.ufpr.br/>.

Para acessar as funções de uma biblioteca, é necessário carregá-la com `library(...)`:

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.5
v forcats    1.0.0      v stringr    1.5.1
v ggplot2    3.5.2      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.1
v purrr      1.0.4
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

O tidyverse é uma coleção de pacotes para ciência de dados que compartilham uma filosofia de design.

Ele inclui pacotes para manipulação de dados (`dplyr`, `tidyr`), visualização (`ggplot2`) e muito mais.

Suas funções para manipulação de data frames (aqui chamados de tibbles), como `filter`, `select` e `mutate`, facilitam a análise de dados tabulares:

```
alunos <- tibble(
  nome = c("João", "Maria", "Pedro"),
  idade = c(30, 25, 28),
  nota_final = c(8.5, 9.0, 6.5),
  turma = c("A", "B", "A")
)

alunos_filtrados <- alunos |>
  filter(idade > 25) |> # filtrar os alunos com mais de 25 anos
  mutate(aprovado = nota_final >= 7) |> # nova coluna "aprovado" com base na nota final
  select(nome, idade, aprovado) # selecionar apenas as colunas nome e idade

print(alunos_filtrados)
```

```
# A tibble: 2 x 3
  nome  idade aprovado
<chr> <dbl> <lgl>
1 João    30 TRUE
2 Pedro   28 FALSE
```

Calculando a média de idade dos alunos aprovados:

```
media_idade_aprovados <- alunos |>
  filter(nota_final >= 7) |>
  summarise(media_idade = mean(idade))

print(media_idade_aprovados)
```

```
# A tibble: 1 x 1
  media_idade
      <dbl>
1      27.5
```

Em resumo, vimos exemplos de funções:

- `filter()`: filtra linhas com base em condições lógicas
- `mutate()`: cria novas colunas ou modifica colunas existentes
- `select()`: seleciona colunas específicas
- `summarise()`: calcula estatísticas resumidas, como média, soma, etc

Existem várias outras funções para manipulação de data frames incluídas no **tidyverse**. Um resumo visual dessas funções está disponível em <https://nyu-cdsc.github.io/learningr/assets/data-transformation.pdf>.

1.3.9 Strings (cadeias de caracteres)

String é o tipo de dado utilizado para representar textos. Em R, strings são definidas com aspas simples ou duplas:

```
texto <- "Olá, mundo!"  
  
print(texto)
```

```
[1] "Olá, mundo!"
```

A biblioteca **stringr** (parte do **tidyverse**) possui diversas funções para a manipulação de strings (todas começam com o prefixo **str_**):

```
a <- "olá"  
b <- 'mundo'  
a_b_concatenados <- str_c(a, b, sep = " ")  
  
print(a_b_concatenados)
```

```
[1] "olá mundo"
```

- Concatenar significa unir duas ou mais strings. A função `str_c` do pacote **stringr** é usada para esta finalidade, e o argumento **sep** especifica o caractere separador a ser inserido entre as strings (neste caso, um espaço).

```
num_palavras <- str_length("quantos caracteres tem esta frase?")  
  
print(num_palavras)
```

[1] 34

- A função `str_length` retorna o número de caracteres em uma string, incluindo espaços e pontuação.

```
contem_texto <- str_detect("este texto contém palavras", "texto")  
  
print(contem_texto)
```

[1] TRUE

- A função `str_detect` verifica se uma string contém um padrão de texto. Ela retorna TRUE se o padrão for encontrado e FALSE caso contrário.

As funções do pacote `stringr` são vetorizadas, o que significa que elas podem ser aplicadas diretamente a um vetor de textos, retornando um resultado para cada elemento.

```
discursos <- c(  
  "este texto contém palavras",  
  "este outro texto não tem a palavra-chave",  
  "a palavra-chave está presente aqui"  
)  
contem_texto <- str_detect(discursos, "palavra-chave")  
  
print(contem_texto)
```

[1] FALSE TRUE TRUE

```
print(discursos[contem_texto])
```

```
[1] "este outro texto não tem a palavra-chave"  
[2] "a palavra-chave está presente aqui"
```

1.4 Exemplo: Análise de Sentimentos com R

```
library(tidyverse)
```

Criando um vetor de textos:

```

textos <- c(
  "Eu amo este produto, é incrível!",
  "Este serviço é péssimo, estou muito insatisfeito.",
  "O filme foi ótimo, adorei a história e os personagens.",
  "A comida estava horrível, não recomendo este restaurante.",
  "O atendimento foi excelente, muito atenciosos e rápidos.",
  "Não volto mais a este lugar."
)

print(textos)

```

```

[1] "Eu amo este produto, é incrível!"
[2] "Este serviço é péssimo, estou muito insatisfeito."
[3] "O filme foi ótimo, adorei a história e os personagens."
[4] "A comida estava horrível, não recomendo este restaurante."
[5] "O atendimento foi excelente, muito atenciosos e rápidos."
[6] "Não volto mais a este lugar."

```

Criando um data frame com os textos:

```

df_textos <- tibble(texto = textos)

print(df_textos)

```

```

# A tibble: 6 x 1
  texto
  <chr>
1 Eu amo este produto, é incrível!
2 Este serviço é péssimo, estou muito insatisfeito.
3 O filme foi ótimo, adorei a história e os personagens.
4 A comida estava horrível, não recomendo este restaurante.
5 O atendimento foi excelente, muito atenciosos e rápidos.
6 Não volto mais a este lugar.

```

Criando um vetor de palavras positivas e negativas:

```

palavras_positivas <- c("amo", "incrível", "ótimo",
                        "adorei", "excelente", "atenciosos", "rápidos")
palavras_negativas <- c("péssimo", "insatisfeito", "horrível",
                        "não recomendo", "ruim", "lento")

```

```
print(palavras_positivas)
```

```
[1] "amo"          "incrível"    "ótimo"       "adorei"      "excelente"
[6] "atenciosos"  "rápidos"
```

```
print(palavras_negativas)
```

```
[1] "péssimo"      "insatisfeito" "horrível"     "não recomendo"
[5] "ruim"         "lento"
```

Criando uma função para analisar o sentimento de um texto:

```
analisar_sentimento <- function(palavras) {
  palavras_positivas_encontradas <- sum(palavras %in% palavras_positivas)
  palavras_negativas_encontradas <- sum(palavras %in% palavras_negativas)
  if (palavras_positivas_encontradas > palavras_negativas_encontradas) {
    return("Positivo")
  } else if (palavras_negativas_encontradas > palavras_positivas_encontradas) {
    return("Negativo")
  } else {
    return("Neutro")
  }
}

print(analisar_sentimento(c("este", "produto", "é", "incrível")))
```

```
[1] "Positivo"
```

Aplicando a função de análise de sentimento a cada texto:

```
df_analise <- df_textos |>
  mutate(texto_min = str_to_lower(texto)) |> # converter para minúsculas
  mutate(palavras = str_extract_all(texto_min, boundary("word")))|> # dividir o texto em palavras
  mutate(sentimento = map_chr(palavras, analisar_sentimento)) |> # map_chr aplica a função
  select(texto, sentimento) # selecionar apenas as colunas texto e sentimento

print(df_analise)
```



```
# A tibble: 6 x 2
  texto                                sentimento
  <chr>                                <chr>
1 Eu amo este produto, é incrível!      Positivo
2 Este serviço é péssimo, estou muito insatisfeito. Negativo
3 O filme foi ótimo, adorei a história e os personagens. Positivo
4 A comida estava horrível, não recomendo este restaurante. Negativo
5 O atendimento foi excelente, muito atenciosos e rápidos. Positivo
6 Não volto mais a este lugar.          Neutro
```

```
df_contagem <- df_analise |>
  group_by(sentimento) |> # agrupar por sentimento
  summarise(contagem = n()) # contar o número de textos em cada categoria de sentimento

print(df_contagem)
```

```
# A tibble: 3 x 2
  sentimento contagem
  <chr>          <int>
1 Negativo         2
2 Neutro           1
3 Positivo         3
```

1.5 Para saber mais

As três primeiras partes do livro *R for Data Science* (Wickham 2023) são uma ótima introdução à linguagem R e à manipulação de dados com a biblioteca `tidyverse`.

1.6 Atividade prática

1. Crie um vetor de textos contendo pelo menos 5 frases de discursos políticos.
2. Crie um data frame com esses textos.
3. Defina 3 vetores com palavras representando temas políticos (ex: economia, saúde, educação).
4. Crie uma função que analise um texto, identifique a qual tema ele mais se relaciona e retorne o tema predominante.
5. Aplique a função de análise de temas a cada texto e crie uma nova coluna no data frame indicando o tema predominante em cada texto.

2 Palavras, morfemas e tokens

i Nesta aula

- Fontes de dados textuais para política.
- O conceito de **Corpus** e a biblioteca *quanteda*.
- Níveis de análise: de caracteres a discursos.
- Processamento básico: Tokenização, *Stopwords* e *Stemming*.
- Exploração inicial: *Keyword-in-context* (KWIC) e Nuvens de Palavras.

```
# instalando pacotes utilizados nesta aula
if (!"tidyverse" %in% installed.packages()) {
  install.packages("tidyverse")
}
if (!"quanteda" %in% installed.packages()) {
  install.packages("quanteda")
}
if (!"quanteda.textstats" %in% installed.packages()) {
  install.packages("quanteda.textstats")
}
if (!"quanteda.textplots" %in% installed.packages()) {
  install.packages("quanteda.textplots")
}
if (!"stopwords" %in% installed.packages()) {
  install.packages("stopwords")
}
```

2.1 Obtendo dados textuais

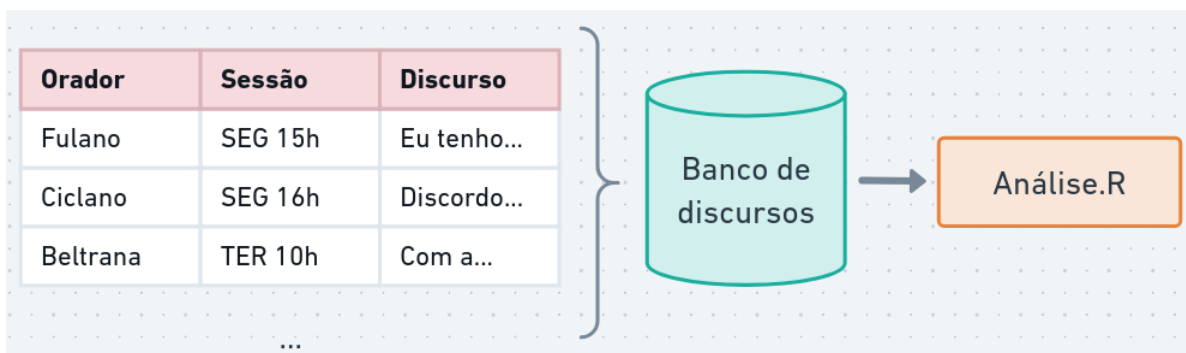
Para realizar uma análise de discurso mediada por computador, o primeiro passo é transformar o material empírico (discursos, leis, posts em redes sociais) em um formato que o computador consiga processar.



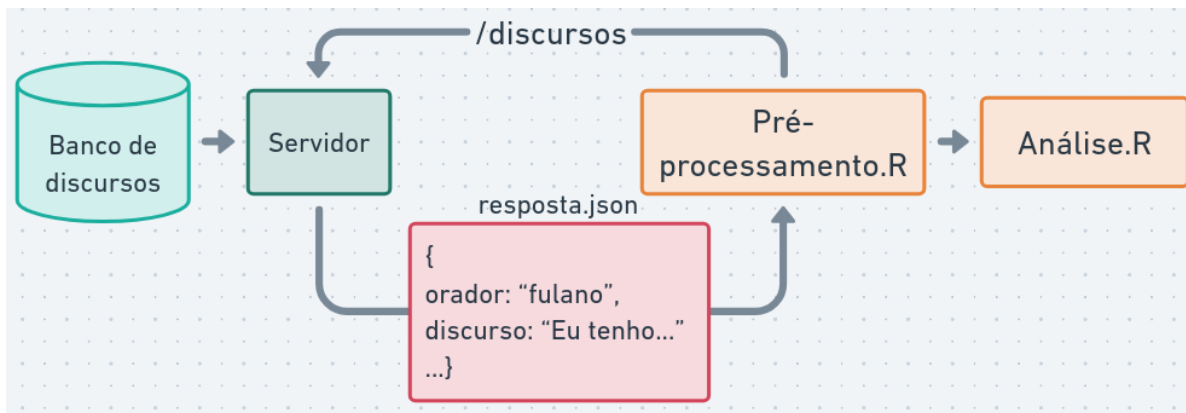
Quais são as fontes de textos interessantes para estudos políticos?

2.1.1 Tipos de fontes de dados textuais

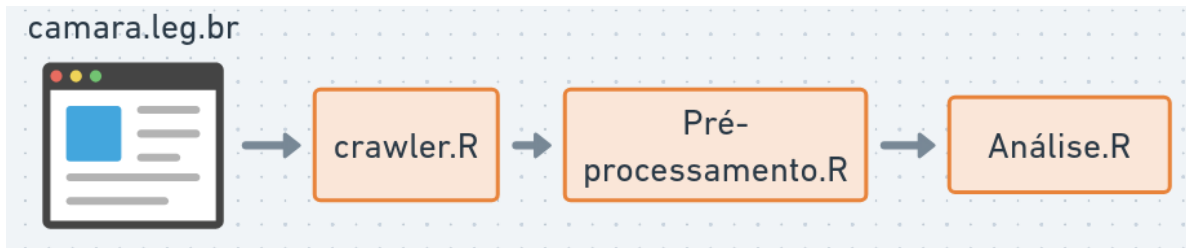
- **Banco de dados e APIs:** Muitas instituições (como a Câmara dos Deputados) oferecem **APIs**, que permitem baixar dados automaticamente com **metadados** valiosos: quem falou, quando, partido, etc.



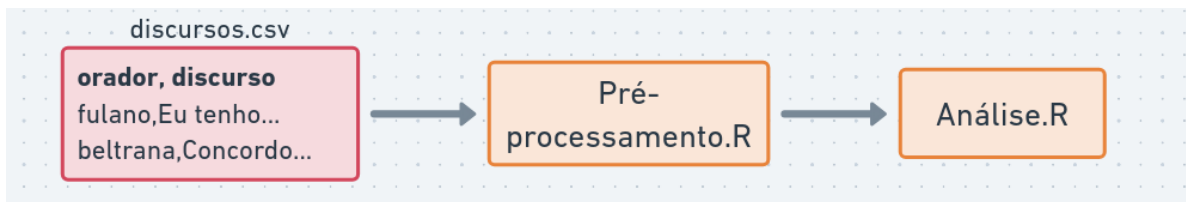
APIs permitem fazer requisições e coletar dados semi-estruturados (JSON ou XML).



Web scraping e OCR: Técnicas para extrair textos diretamente de sites ou PDFs. Útil quando os dados não estão organizados em bancos.



Arquivos estruturados (CSV, Excel): Forma comum de trabalhar com dados já coletados.



2.1.2 Exemplo: Carregando Discursos da Câmara

Nesta aula, para focar na análise textual, utilizaremos um conjunto de dados já extraído da API da Câmara dos Deputados, contendo discursos de parlamentares organizados em um arquivo CSV.

Para manipular esses dados, utilizaremos o pacote **tidyverse**, que é o padrão para organização de dados na linguagem R.

```

library(tidyverse)

# Carregando os dados
df_discursos <- read_csv("discursos.csv")

# Visualizando as primeiras linhas e colunas mais relevantes
df_discursos |> select(nome, partido, uf, transcricao) |> head()

# A tibble: 6 x 4
  nome          partido    uf  transcricao
  <chr>         <chr>    <chr> <chr>
1 Acácio Favacho MDB      AP  "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. ~
2 Acácio Favacho MDB      AP  "O SR. ACÁCIO FAVACHO (Bloco/MDB - AP. Sem~
3 Acácio Favacho MDB      AP  "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. ~
4 Adail Filho    REPUBLICANOS AM  "O SR. ADAIL FILHO (Bloco/REPUBLICANOS - A~
5 Adilson Barroso PL      SP  "O SR. ADILSON BARROSO (Bloco/PL - SP. Sem~
6 Adilson Barroso PL      SP  "O SR. ADILSON BARROSO (Bloco/PL - SP. Sem~
  
```

2.1.3 Criando o corpus

Na análise de textos, chamamos de **corpus** o conjunto de documentos que pretendemos estudar. Um corpus não é apenas uma “pilha de textos”, mas uma coleção organizada que preserva o vínculo entre o texto e seus **metadados** (as variáveis que contextualizam a fala, como o orador ou o partido).

Utilizaremos a biblioteca **quanteda**, uma das mais robustas para análise quantitativa de textos.

```
library(quanteda)

# Criando um identificador único para cada discurso
df_discursos <- df_discursos |>
  mutate(doc_id = str_c(row_number(), nome, partido, sep = "_"))

# Criando o objeto de corpus
corpus_discursos <- corpus(df_discursos,
  text_field = "transcricao",
  docid_field = "doc_id")

summary(corpus_discursos, n = 5)
```

Corpus consisting of 1849 documents, showing 5 documents:

	Text	Types	Tokens	Sentences	id	nome	uf
1_Acácio Favacho_MDB	256	428	16	204379	Acácio Favacho	AP	
2_Acácio Favacho_MDB	301	659	42	204379	Acácio Favacho	AP	
3_Acácio Favacho_MDB	318	588	29	204379	Acácio Favacho	AP	
4_Adail Filho_REPUBLICANOS	163	260	13	220714	Adail Filho	AM	
5_Adilson Barroso_PL	243	501	19	221328	Adilson Barroso	SP	
partido					uri		
MDB	https://dadosabertos.camara.leg.br/api/v2/deputados/204379						
MDB	https://dadosabertos.camara.leg.br/api/v2/deputados/204379						
MDB	https://dadosabertos.camara.leg.br/api/v2/deputados/204379						
REPUBLICANOS	https://dadosabertos.camara.leg.br/api/v2/deputados/220714						
PL	https://dadosabertos.camara.leg.br/api/v2/deputados/221328						
	nome_civil	sexo	cpf	data_nascimento			
ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28				
ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28				
ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28				
ADAIL JOSÉ FIGUEIREDO PINHEIRO	M	77267796249	1992-02-16				

ADILSON BARROSO OLIVEIRA		M 05585378805		1964-06-14	
data_falecimento	uf_nascimento	municipio_nascimento	escolaridade		
NA	AP	Macapá	Superior		
NA	AP	Macapá	Superior		
NA	AP	Macapá	Superior		
NA	AM	Manaus	Superior	Incompleto	
NA	MG	Minas Novas	Superior		
situacao	data_situacao	ultimo_partido	hora_inicio	fase_evento	
Exercício	2023-02-01	MDB	2025-04-29 13:55:00	Encerramento	
Exercício	2023-02-01	MDB	2025-04-02 10:32:00	Homenagem	
Exercício	2023-02-01	MDB	2025-04-01 13:55:00	Encerramento	
Exercício	2023-02-01	REPUBLICANOS	2025-04-02 14:44:00	Breves Comunicações	
Exercício	2025-12-14	PL	2025-04-22 17:16:00	Breves Comunicações	
tipo_discurso					
DISCURSO ENCAMINHADO					
HOMENAGEM					
DISCURSO ENCAMINHADO					
PELA ORDEM					
BREVES COMUNICAÇÕES					

Medida provisória, Crédito extraordinário, Comunidade ribeirinha, Crise climática, Amazonas, Seguros, Defesa, Pescador, Biodiversidade, Família, vulnerabilidade, Governo federal, Ministra de Estado, Minas

O Deputado relatou
O Deputado discursou na sessão

O Deputado destacou a importância da aprovação da Medida Provisória 1.362, que trata da criação do Parque Nacional de Defesa aos pescadores, beneficiando famílias vulneráveis e protegendo a biodiversidade local.
O Deputado fez críticas ao Supremo Tribunal Federal, afirmando que parte dos Ministros teria

As informações extras (metadados) ficam armazenadas como **docvars** (variáveis de documento).

```
docvars(corpus_discursos) |> head()
```

	id	nome	uf	partido
1	204379	Acácio Favacho	AP	MDB
2	204379	Acácio Favacho	AP	MDB

3	204379	Acácio Favacho	AP	MDB
4	220714	Adail Filho	AM	REPUBLICANOS
5	221328	Adilson Barroso	SP	PL
6	221328	Adilson Barroso	SP	PL

uri

1	https://dadosabertos.camara.leg.br/api/v2/deputados/204379
2	https://dadosabertos.camara.leg.br/api/v2/deputados/204379
3	https://dadosabertos.camara.leg.br/api/v2/deputados/204379
4	https://dadosabertos.camara.leg.br/api/v2/deputados/220714
5	https://dadosabertos.camara.leg.br/api/v2/deputados/221328
6	https://dadosabertos.camara.leg.br/api/v2/deputados/221328

	nome_civil	sexo	cpf	data_nascimento
1	ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28
2	ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28
3	ACÁCIO DA SILVA FAVACHO NETO	M	74287028287	1983-09-28
4	ADAIL JOSÉ FIGUEIREDO PINHEIRO	M	77267796249	1992-02-16
5	ADILSON BARROSO OLIVEIRA	M	05585378805	1964-06-14
6	ADILSON BARROSO OLIVEIRA	M	05585378805	1964-06-14

	data_falecimento	uf_nascimento	municipio_nascimento	escolaridade
1	NA	AP	Macapá	Superior
2	NA	AP	Macapá	Superior
3	NA	AP	Macapá	Superior
4	NA	AM	Manaus	Superior Incompleto
5	NA	MG	Minas Novas	Superior
6	NA	MG	Minas Novas	Superior

	situacao	data_situacao	ultimo_partido	hora_inicio
1	Exercício	2023-02-01	MDB	2025-04-29 13:55:00
2	Exercício	2023-02-01	MDB	2025-04-02 10:32:00
3	Exercício	2023-02-01	MDB	2025-04-01 13:55:00
4	Exercício	2023-02-01	REPUBLICANOS	2025-04-02 14:44:00
5	Exercício	2025-12-14	PL	2025-04-22 17:16:00
6	Exercício	2025-12-14	PL	2025-04-09 16:00:00

	fase_evento	tipo_discurso
1	Encerramento	DISCURSO ENCAMINHADO
2	Homenagem	HOMENAGEM
3	Encerramento	DISCURSO ENCAMINHADO
4	Breves Comunicações	PELA ORDEM
5	Breves Comunicações	BREVES COMUNICAÇÕES
6	Breves Comunicações	BREVES COMUNICAÇÕES

1
2
3

4 Medida provisória,Crédito extraordinário,Comunidade ribeirinha,Crise climática,Amazonas,Se
 defeso,Pescador,Biodiversidade,Família,vulnerabilidade,Governo federal,Ministra de Estado,Min
 5
 6 justificat
 presidente da República,Crescimento econômico,Imposto,Mérito,Voto contrário

1 O Deputado relat
 2 O Deputado discursou na sessão s
 3
 4 O Deputado destacou a importância da aprovação da Medida
 defeso aos pescadores, beneficiando famílias vulneráveis e protegendo a biodiversidade local
 5 O Deputado fez críticas ao Supremo Tribunal Federal, afirmando que parte dos Ministros ter
 6
 Presidente Bolsonaro e a necessidade de estimular o crescimento econômico por meio de menos t

Podemos filtrar o corpus com base nelas, o que é essencial para análises comparativas em
 ciência política:

```
# Exemplo: Criar um sub-corpus apenas com discursos de parlamentares do PT
corpus_pt <- corpus_subset(corpus_discursos, partido == "PT")
print(corpus_pt)
```

Corpus consisting of 373 documents and 21 docvars.

39_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA. Sem revisão do orador.)..."

40_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA. Sem revisão do orador.)..."

41_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA) - Sr. Presidente, em no..."

42_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA. Pela ordem. Sem revisão..."

43_Airton Faleiro_PT :

"O SR. AIRTON FALEIRO (Bloco/PT - PA. Pela ordem. Sem revisão..."

57_Alencar Santana_PT :

"O SR. ALENCAR SANTANA (Bloco/PT - SP. Sem revisão do orador...."

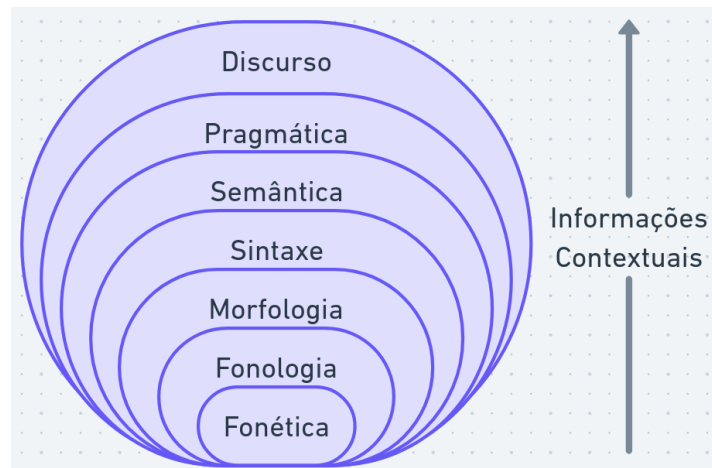
[reached max_ndoc ... 367 more documents]

Outras bibliotecas relacionadas que vamos utilizar são: `quanteda.textstats`, para gerar estatísticas, e `quanteda.textplots`, para visualizações gráficas.

```
library(quanteda.textstats)
library(quanteda.textplots)
```

2.2 Níveis de análise textual

O estudo da língua é dividido em diversas subáreas:



- **fonética** e **fonologia**: sons e sua organização
- **morfologia**: como morfemas se organizam para formar palavras
- **sintaxe**: como palavras se organizam para formar sintagmas e orações
- **semântica**: significado de palavras e frases
- **pragmática**: organização de orações para fins comunicativos
- **discurso**: estruturas do texto como todo

O processamento de textos pelo computador (PLN) foca principalmente no texto escrito (sintaxe, semântica e discurso). Contudo, a fala e a entonação também são vitais. Dada a natureza multimodal da análise de discurso, as marcas de oralidade carregam forte peso interpretativo.

Observe o exemplo abaixo das notas taquigráficas da Câmara:

Que tipo de informação é transmitida por meio da fala, além do conteúdo semântico? A transcrição em texto consegue capturar sarcasmo, exaltação ou hesitação? Perceba como os taquígrafos utilizam parênteses e marcadores para tentar registrar reações do plenário.

2.3 A estrutura das palavras: morfemas, lemas e radicais

A forma como dividimos o texto depende do objetivo da nossa análise. Separar palavras em subunidades pode ser útil para lidar com: - neologismos, palavras incorretas ou desconhecidas; - complexidade do texto ou dialetos; - gêneros literários alternativos.

Por exemplo, Moffitt (2016) destaca o uso de neologismos em performances populistas. Na morfologia, a unidade mínima de significado é o **morfema** (ex: “casinhas” = casa + inha + s).

2.3.1 Lematização e Anotação Morfossintática

A **lematização** é o processo de reduzir uma palavra à sua forma original de dicionário (seu “lema”). Por exemplo, “gatos” e “gata” são convertidos para “gato”, e “fui” é convertido para o verbo “ir”.

No português, esse processo é complexo e exige dicionários e ferramentas avançadas (como o projeto MorphoBR ou a biblioteca **spacyr**), pois depende do contexto (ex: “canto” pode ser o verbo cantar ou o substantivo). Isso enriquece a análise de discurso ao agrupar flexões verbais e nominais sob um mesmo guarda-chuva.

2.3.2 Stemização (Corte do radical)

Uma alternativa mais simples (e crua) muito usada computacionalmente é a **stemização** (Stemming), que é o processo de cortar as desinências da palavra, mantendo apenas o seu radical (raiz).

Isso reduz drasticamente o vocabulário do texto, facilitando análises de frequência, mas as palavras geradas podem ser ambíguas ou perder nuances de gênero/número que seriam relevantes para a análise.

Exemplo utilizando a biblioteca **quanteda**:

```
stems_testar <- char_wordstem(c("testar", "testando", "testarei"), language = "portuguese")
print(stems_testar)
```

```
[1] "test" "test" "test"
```

2.3.3 Caracteres e Codificação

A menor unidade de informação em um texto digital é o **caractere** (ou símbolo). O computador usa padrões, como o **UTF-8**, para catalogar caracteres (incluindo emojis e acentuação gráfica, essenciais em português).

```
print(tokenize_character("#feliz! \U0001f600"))
```

```
[[1]]  
[1] "#" "f" "e" "l" "i" "z" "!" " " " "
```

2.4 Tokenização (Dividindo o texto)

O problema de separar o texto em unidades mínimas para análise é conhecido como **tokenização**. Em geral, quebramos os discursos em palavras, mas podemos tokenizar por frases se o objetivo for analisar sentenças inteiras.

```
texto <- 'A deputada disse: "Não aceitaremos cortes". A oposição concordou.'  
  
# Tokenização removendo pontuação  
tokens_palavras <- tokens(texto, remove_punct = TRUE)  
print(tokens_palavras)
```

Tokens consisting of 1 document.

```
text1 :  
[1] "A" "deputada" "disse" "Não" "aceitaremos"  
[6] "cortes" "A" "oposição" "concordou"
```

```
# Tokenização por sentenças  
tokens_frases <- tokens(texto, what = "sentence")  
print(tokens_frases)
```

Tokens consisting of 1 document.

```
text1 :  
[1] "A deputada disse: \"Não aceitaremos cortes\"."  
[2] "A oposição concordou."
```

2.4.1 A Matriz de Documentos e Termos (DFM)

Para o computador fazer contagens, ele precisa organizar os tokens em uma estrutura chamada **Matriz de Documentos e Termos (Document-Feature Matrix, DFM)**.

Pense no DFM como uma grande planilha: cada **linha** é um discurso e cada **coluna** é uma palavra do vocabulário. A célula contém a quantidade de vezes que aquela palavra apareceu naquele discurso.

```
dfm_discursos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_numbers = TRUE) |>
  tokens_tolower() |> # converter para minúsculo
  dfm()

print(dfm_discursos)
```

Document-feature matrix of: 1,849 documents, 25,831 features (99.29% sparse) and 21 docvars.

docs	features
	discurso na íntegra encaminhado pelo sr deputado
1_Acácio Favacho_MDB	1 4 1 1 1 1 2
2_Acácio Favacho_MDB	0 2 0 0 0 3 1
3_Acácio Favacho_MDB	1 2 1 1 3 1 1
4_Adail Filho_REPUBLICANOS	0 2 0 0 1 2 0
5_Adilson Barroso_PL	0 3 0 0 0 1 0
6_Adilson Barroso_PL	0 5 0 0 0 1 2

docs	features
	acácio favacho sem
1_Acácio Favacho_MDB	1 1 1
2_Acácio Favacho_MDB	1 1 1
3_Acácio Favacho_MDB	1 1 1
4_Adail Filho_REPUBLICANOS	0 0 1
5_Adilson Barroso_PL	0 0 1
6_Adilson Barroso_PL	0 0 2

[reached max_ndoc ... 1,843 more documents, reached max_nfeat ... 25,821 more features]

Por exemplo, para ver a frequência da palavra “presidente”:

```
print(sum(dfm_discursos[, "presidente"]))
```

```
[1] 4737
```

2.5 Stopwords

Stopwords são palavras funcionais muito comuns (como “o”, “a”, “de”, “e”, “mas”) que, isoladamente, não ajudam a diferenciar os temas dos discursos. Elas são frequentemente removidas para reduzir o “ruído” nos gráficos e nas contagens.

No R, podemos usar o pacote `stopwords` para obter listas pré-definidas em português:

```
library(stopwords)
stopwords_portugues <- stopwords(language = "pt")
print(stopwords_portugues[1:10])
```

```
[1] "de"    "a"     "o"     "que"   "e"     "do"    "da"    "em"    "um"    "para"
```

Atenção: A remoção de stopwords é ótima para descobrir os *temas* de um texto, mas pode ser problemática se você estiver analisando *estilos* ou *marcadores conversacionais* (o uso excessivo do pronome “nós” vs “eu”, por exemplo, desapareceria se você os tratasse como stopwords).

2.6 Palavras no contexto (Keywords-in-context)

Keywords-in-context (KWIC) é um método qualitativo excelente para a Análise de Discurso. Ele busca um termo específico e exibe a palavra central cercada pelas palavras que vêm antes e depois (o seu contexto).

Isso nos permite analisar os *enquadramentos* (frames) e *associações* feitas pelos oradores.

```
# Buscando o termo "anistia" e observando 3 palavras de contexto
kwic_anistia <- corpus_discursos |>
  tokens() |>
  kwic("anistia", window = 3) |>
  head(10)

kwic_anistia
```

Keyword-in-context with 10 matches.

[5_Adilson Barroso_PL, 88]	a Lei da Anistia , a extrema
[22_Adriana Ventura_NOVO, 363]	a favor da anistia . E a
[22_Adriana Ventura_NOVO, 374]	a favor da anistia não é passar
[39_Airton Faleiro_PT, 73]	que é a anistia . Eu,
[39_Airton Faleiro_PT, 395]	total, querem anistia total. Eu

[39_Airton Faleiro_PT, 619]	principais não têm	anistia	e nem têm
[39_Airton Faleiro_PT, 650]	no Brasil.	Anistia	, não!
[55_Alberto Fraga_PL, 361]	falar aqui de	anistia	, porque sou
[60_Alencar Santana_PT, 1081]	não" à	anistia	. Alguns setores
[60_Alencar Santana_PT, 1105]	que votar a	anistia	. Nós não

Você também pode usar o curinga * para buscar variações (ex: “golp*” pegará “golpe” e “golpista”):

```
golp_kwic <- corpus_discursos |>
  tokens() |>
  kwic("golp*", window = 3) |>
  head(10)

golp_kwic
```

Keyword-in-context with 10 matches.

[39_Airton Faleiro_PT, 255]	quem idealizou o	golpe
[39_Airton Faleiro_PT, 262]	a mobilização do	golpe
[39_Airton Faleiro_PT, 352]	, orquestrou o	golpe
[39_Airton Faleiro_PT, 444]	quem executou o	golpe
[39_Airton Faleiro_PT, 474]	impedir que esse	golpe
[39_Airton Faleiro_PT, 534]	, executa um	golpe
[39_Airton Faleiro_PT, 562]	tentar dar um	golpe
[44_Alberto Fraga_PL, 124]	preparando mais um	golpe
[62_Alencar Santana_PT, 29]	Prisão para os	golpistas
[62_Alencar Santana_PT, 52]	Brasil sofreu um	golpe

, quem financiou
 , quem praticou
 contra a democracia
 contra a democracia
 se efetivasse,
 de Estado,
 , ocupar e
 . Isso aqui
 . É isso
 , praticado pelos

2.7 Exemplo Prático: Nuvem de Palavras por Partido

Vamos consolidar tudo que aprendemos criando uma visualização que compara as palavras mais ditas pelos deputados do PT e do PL.

Passo 1: Tokenização e Limpeza Vamos quebrar em palavras, remover pontuação, converter para minúsculo e remover stopwords do português.

```
tokens_limpos <- corpus_discursos |>
  tokens(remove_punct = TRUE, remove_symbols = TRUE, remove_numbers = TRUE) |>
  tokens_tolower() |>
  tokens_remove(stopwords(language = "pt"))
```

Passo 2: Criando o DFM e removendo jargões políticos O plenário da Câmara é cheio de vícios de linguagem (“senhor”, “presidente”, “deputado”). Se não os removermos, a nossa análise será engolida por eles.

```
dfm_final <- dfm(tokens_limpos)

# Criando lista de palavras "irrelevantes" para o nosso contexto
palavras_irrelevantes <- c("senhor", "sr", "senhora", "sra", "sobre", "presidente",
                           "deputado", "deputada", "lei", "nº", "projeto", "é", "nesta", "de")

dfm_final <- dfm_final |> dfm_remove(palavras_irrelevantes)
```

Passo 3: Visualizando a Diferença (Wordcloud) Vamos filtrar os partidos de interesse, agrupar o DFM por partido e gerar a nuvem de comparação. As palavras maiores representam os termos mais frequentes.

```
dfm_final |>
  dfm_subset(partido %in% c("PT", "PL")) |>      # filtrar partidos
  dfm_group(groups = partido) |>                # agrupar matriz por partido
  textplot_wordcloud(comparison = TRUE,          # modo comparativo
                     color = c("red", "blue"),  # cores PT (red) e PL (blue)
                     min_count = 5,
                     max_words = 150)
```

PL



PT

2.8 Para saber mais

- Leia mais sobre morfemas, tokenização no Capítulo 2 do livro “Speech and Language Processing” (Jurafsky and Martin 2026) e os Capítulos 4 e 5 do livro “Processamento de Linguagem Natural” (Caseli and Nunes 2024).
- Estude os tutoriais completos da biblioteca `quanteda` em <https://tutorials.quanteda.io/>. Saiba mais sobre a biblioteca lendo o artigo de Benoit et al. (2018).

2.9 Atividade prática

1. Carregue o arquivo `discursos.csv` e familiarize-se com as colunas.
2. Crie um *corpus* filtrando apenas os discursos de deputados de um gênero ou estado.
3. Realize a tokenização e remova as *stopwords*.
4. Crie uma Matriz de Documentos e Termos (DFM).
5. Conte as palavras mais comuns utilizando `textstat_frequency` e visualize os resultados utilizando uma nuvem de palavras.
6. *Desafio*: Tente utilizar a função `kwic` para descobrir em que contexto os parlamentares do gênero ou estado utilizaram a palavra “saúde” ou “educação”.

Glossário

Introdução

ADP Análise de Discurso Político

PLN (NLP) Processamento de Linguagem Natural (Natural Language Processing)

Aula 1

R linguagem de programação utilizada para análise de dados

RStudio ambiente de desenvolvimento integrado (IDE) para R

Tidyverse coleção de pacotes R para ciência de dados

Variável nome para um valor armazenado em memória, que pode ser alterado

Condicional estrutura de código que executa ou não um bloco, de acordo com uma expressão lógica

Função um trecho de código que possui um nome, geralmente para uso de operações comuns

|> operador pipe utilizado para encadear chamadas de funções (o resultado é passado como argumento para a próxima função)

Vetor coleção ou sequência de um ou mais valores do mesmo tipo

Data Frame estrutura de dado para armazenar tabelas com linhas e colunas

Biblioteca coleção de funções, pode ser instalada e compartilhada em repositórios na Internet

String cadeia de caracteres, tipo de dado utilizado para representar texto

Referências

- Benoit, Kenneth, Kohei Watanabe, Haiyan Wang, Paul Nulty, Adam Obeng, Stefan Müller, and Akitaka Matsuo. 2018. “Quanteda: An r Package for the Quantitative Analysis of Textual Data.” *Journal of Open Source Software* 3 (30): 774. <https://doi.org/10.21105/joss.00774>.
- Caseli, H. M., and M. G. V. Nunes, eds. 2024. *Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações Em Português*. 2nd ed. BPLN. <https://brasileiraspln.com/livro-pln/2a-edicao/>.
- Jurafsky, Daniel, and James H. Martin. 2026. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition, with Language Models*. 3rd ed. <https://web.stanford.edu/~jurafsky/slp3/>.
- Moffitt, Benjamin. 2016. *The Global Rise of Populism: Performance, Political Style, and Representation*. 1st ed. Stanford University Press. <https://doi.org/10.2307/j.ctvqsdsd8>.
- O’Neill, Lachlan, Nandini Anantharama, Wray Buntine, and Simon D. Angus. 2021. “Quantitative Discourse Analysis at Scale - AI, NLP and the Transformer Revolution.” *SoDa Laboratories Working Paper Series*, SoDa Laboratories Working Paper Series, December. <https://ideas.repec.org/p/ajr/sodwps/2021-12.html>.
- Silge, Julia, and David Robinson. 2017. *Text mining with R: a tidy approach*. Beijing, China: O’Reilly.
- Wickham, Hadley. 2023. *R for Data Science*. 2nd ed. Sebastopol: O’Reilly Media, Incorporated.

A Coletando Dados do legislativo

A.1 Coletando dados da API da Câmara dos Deputados

A Câmara dos Deputados disponibiliza dados de discursos por meio de um sistema automatizado (API).

A documentação da API (versão 2) está disponível em: <https://dadosabertos.camara.leg.br/>

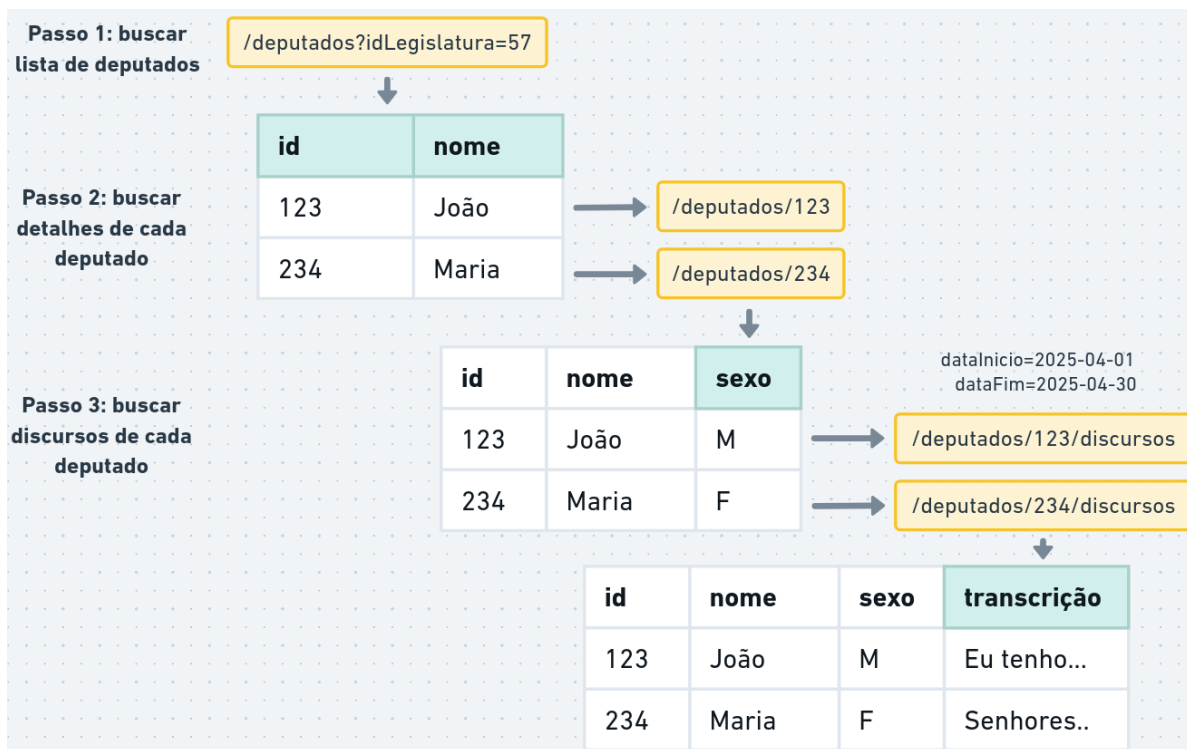
Para baixar e organizar esses dados, utilizamos pacotes da linguagem R:

- `httr2` para fazer requisições na API
- `tidyverse` para organizar e tratar os dados

```
if (!"tidyverse" %in% installed.packages()) {  
  install.packages("tidyverse")  
}  
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.1.4      v readr      2.1.5  
v forcats    1.0.0      v stringr    1.5.1  
v ggplot2    3.5.2      v tibble     3.3.0  
v lubridate  1.9.4      v tidyr      1.3.1  
v purrr      1.0.4  
-- Conflicts ----- tidyverse_conflicts() --  
x dplyr::filter() masks stats::filter()  
x dplyr::lag()     masks stats::lag()  
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
if (!"httr2" %in% installed.packages()) {  
  install.packages("httr2")  
}  
library(httr2)
```



Primeiro, vamos montar uma requisição (função `request`) para obter dados dos deputados na legislatura 57 (atual):

```
url_api <- "https://dadosabertos.camara.leg.br/"

requisicao_deputados <- request(url_api) |>
  req_url_path("api", "v2", "deputados") |>
  req_url_query(
    idLegislatura = 57, # número da legislatura
    nome = "",        # parte do nome
    siglaUF = "",     # sigla UF eleito
    siglaPartido = "", # sigla do partido
    siglaSexo = "",   # sexo (M ou F)
  )

print(requisicao_deputados)
```

<http2_request>

GET https://dadosabertos.camara.leg.br/api/v2/deputados?idLegislatura=57&nome=&siglaUF=&siglaPartido=&siglaSexo=

Body: empty

Executamos a requisição (`req_perform`) para obter a resposta do serviço de dados abertos:

```
resp_deputados <- req_perform(requisicao_deputados)

print(resp_deputados)
```

```
<httr2_response>
GET https://dadosabertos.camara.leg.br/api/v2/deputados?idLegislatura=57&nome=&siglaUF=&siglaPartido=
Status: 200 OK
Content-Type: application/json
Body: In memory (248328 bytes)
```

Os dados retornados estão no formato JSON, que é representado no R como uma lista aninhada (lista com elementos de lista).

```
dados_resposta <- resp_body_json(resp_deputados)
dados_resposta <- dados_resposta$dados # acessa elemento "dados" da lista

print(length(dados_resposta)) # número de elementos
```

```
[1] 722
```

```
print(dados_resposta[[1]]) # primeiro elemento
```

```
$id
```

```
[1] 220593
```

```
$uri
```

```
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/220593"
```

```
$nome
```

```
[1] "Abilio Brunini"
```

```
$siglaPartido
```

```
[1] "PL"
```

```
$uriPartido
```

```
[1] "https://dadosabertos.camara.leg.br/api/v2/partidos/37906"
```

```
$siglaUf
```

```
[1] "MT"
```

```
$idLegislatura
```

```
[1] 57
```

```
$urlFoto
```

```
[1] "https://www.camara.leg.br/internet/deputado/bandep/220593.jpg"
```

```
$email
```

```
NULL
```

- `$dados` acessa o elemento “dados” da lista, `[[1]]` acessa o primeiro elemento da lista.

Transformamos a lista em dataframe com o código comentado abaixo:

```
df_deputados_partido <- dados_resposta |>
  tibble() |>                                # transformar em dataframe
  unnest_wider("dados_resposta") |>          # desaninhar em colunas
  select(                                     # seleciona colunas relevantes
    id,
    nome,
    uf = siglaUf,
    partido = siglaPartido,
    uri
  )

head(df_deputados_partido) # head mostra as primeiras linhas do data frame
```

```
# A tibble: 6 x 5
```

	id	nome	uf	partido	uri
	<int>	<chr>	<chr>	<chr>	<chr>
1	220593	Abilio Brunini	MT	PL	https://dadosabertos.camara.leg.br/~
2	204379	Acácio Favacho	AP	MDB	https://dadosabertos.camara.leg.br/~
3	220714	Adail Filho	AM	REPUBLICANOS	https://dadosabertos.camara.leg.br/~
4	221328	Adilson Barroso	SP	PL	https://dadosabertos.camara.leg.br/~
5	204560	Adolfo Viana	BA	PSDB	https://dadosabertos.camara.leg.br/~
6	204528	Adriana Ventura	SP	NOVO	https://dadosabertos.camara.leg.br/~

Note que existem repetições, quando um deputado pertenceu a mais de um partido:

```
deputados_multiplos_partidos <- df_deputados_partido |>
  group_by(nome) |> # agrupar por nome
  filter(n() > 1)   # filtrar grupos com mais de um elemento

head(deputados_multiplos_partidos)
```

```
# A tibble: 6 x 5
# Groups:   nome [3]
      id nome          uf partido uri
  <int> <chr>         <chr> <chr> <chr>
1 220542 Alexandre Guimarães TO REPUBLICANOS https://dadosabertos.camara.leg~
2 220542 Alexandre Guimarães TO MDB          https://dadosabertos.camara.leg~
3 178881 Aluisio Mendes      MA PSC          https://dadosabertos.camara.leg~
4 178881 Aluisio Mendes      MA REPUBLICANOS https://dadosabertos.camara.leg~
5 220612 Bebeto              RJ PTB          https://dadosabertos.camara.leg~
6 220612 Bebeto              RJ PP           https://dadosabertos.camara.leg~
```

A coluna uri contém o endereço para buscarmos detalhes sobre o deputado, como sexo, data de nascimento, situação atual, etc.

Para cada deputado, vamos fazer a requisição que retorna os detalhes. Para isso, vamos mapear cada elemento da coluna uri em uma requisição:

```
df_deputados_detalhes <- df_deputados_partido |>
  mutate(req_detalhes = map(uri, request)) |> # montar requisições
  mutate(resp_detalhes = req_perform_parallel(req_detalhes)) # buscar detalhes e armazenar a
```

```
[working] (699 + 0) -> 10 -> 13 | =>----- 2%
[working] (658 + 0) -> 10 -> 54 | ==>----- 7%
[working] (614 + 0) -> 10 -> 98 | ===>----- 14%
[working] (570 + 0) -> 10 -> 142 | =====>----- 20%
[working] (524 + 0) -> 10 -> 188 | =====>----- 26%
[working] (486 + 0) -> 10 -> 226 | =====>----- 31%
[working] (447 + 0) -> 9 -> 266 | =====>----- 37%
```

[working] (405 + 0) -> 10 -> 307 | =====>----- 43%

[working] (360 + 0) -> 10 -> 352 | =====>----- 49%

[working] (321 + 0) -> 10 -> 391 | =====>----- 54%

[working] (279 + 0) -> 10 -> 433 | =====>----- 60%

[working] (234 + 0) -> 10 -> 478 | =====>----- 66%

[working] (192 + 0) -> 10 -> 520 | =====>----- 72%

[working] (147 + 0) -> 10 -> 565 | =====>----- 78%

[working] (106 + 0) -> 10 -> 606 | =====>----- 84%

[working] (65 + 0) -> 10 -> 647 | =====>---- 90%

Waiting 29s for rate limit =>-----

Waiting 29s for rate limit ===>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>-----

Waiting 29s for rate limit =====>--

Waiting 29s for rate limit =====>

```
[working] (65 + 0) -> 10 -> 647 | =====>--- 90%
[working] (24 + 1) -> 11 -> 687 | =====>- 95%
[working] (0 + 1) -> 0 -> 722 | =====> 100%
```

```
head(df_deputados_detalhes)
```

```
# A tibble: 6 x 7
```

	id	nome	uf	partido	uri	req_detalhes	resp_detalhes
	<int>	<chr>	<chr>	<chr>	<chr>	<list>	<list>
1	220593	Abilio Brunini	MT	PL	https://~	<httr2_rq>	<httr2_rs>
2	204379	Acácio Favacho	AP	MDB	https://~	<httr2_rq>	<httr2_rs>
3	220714	Adail Filho	AM	REPUBLICANOS	https://~	<httr2_rq>	<httr2_rs>
4	221328	Adilson Barroso	SP	PL	https://~	<httr2_rq>	<httr2_rs>
5	204560	Adolfo Viana	BA	PSDB	https://~	<httr2_rq>	<httr2_rs>
6	204528	Adriana Ventura	SP	NOVO	https://~	<httr2_rq>	<httr2_rs>

- `req_perform_parallel` executa múltiplas requisições em paralelo (cada requisição da coluna `req_detalhes`).

Vamos extrair os detalhes da coluna `resp_detalhes` e organizar os dados na tabela.

```
df_deputados <- df_deputados_detalhes |>
mutate(detalhes = map(resp_detalhes, resp_body_json)) |> # ler dados da resposta
unnest_wider(detalhes) |> unnest_wider("dados", names_sep = "_") |> # desaninhar coluna "dados"
unnest_wider("dados_ultimoStatus", names_sep = "_") |> # desaninhar ultimoStatus
select( # selecionar colunas
  # colunas antigas
  colnames(df_deputados_partido),
  # novas colunas
  nome_civil = dados_nomeCivil,
  sexo = dados_sexo,
  cpf = dados_cpf,
  data_nascimento = dados_dataNascimento,
  data_falecimento = dados_dataFalecimento,
  uf_nascimento = dados_ufNascimento,
  municipio_nascimento = dados_municipioNascimento,
  escolaridade = dados_escolaridade,
  situacao = dados_ultimoStatus_situacao,
  data_situacao = dados_ultimoStatus_data,
  ultimo_partido = dados_ultimoStatus_siglaPartido
```

```

) |>
filter(partido == ultimo_partido) |> # considerar apenas último partido
distinct()                          # filtrar dados duplicados

head(df_deputados)

```

A tibble: 6 x 16

```

      id nome      uf partido uri nome_civil sexo cpf data_nascimento
  <int> <chr>    <chr> <chr> <chr> <chr> <chr> <chr> <chr>
1 220593 Abilio Brun~ MT PL http~ ABILIO JA~ M 9977~ 1984-01-31
2 204379 Acácio Fava~ AP MDB http~ ACÁCIO DA~ M 7428~ 1983-09-28
3 220714 Adail Filho AM REPUB~ http~ ADAIL JOS~ M 7726~ 1992-02-16
4 221328 Adilson Bar~ SP PL http~ ADILSON B~ M 0558~ 1964-06-14
5 204560 Adolfo Viana BA PSDB http~ ADOLFO VI~ M 8012~ 1981-02-02
6 204528 Adriana Ven~ SP NOVO http~ ADRIANA M~ F 1251~ 1969-03-06
# i 7 more variables: data_falecimento <chr>, uf_nascimento <chr>,
# municipio_nascimento <chr>, escolaridade <chr>, situacao <chr>,
# data_situacao <chr>, ultimo_partido <chr>

```

Para obter os discursos, vamos definir uma função que monta a requisição para um deputado (por número identificador):

```

req_discursos_deputado <- function(id_deputado) {
  req <- request(url_api) |>
  req_url_path("api", "v2", "deputados", id_deputado, "discursos") |>
  req_url_query(
    dataInicio = "2025-04-01",
    dataFim = "2025-04-30",
    itens=100
  )
  return(req)
}

print(req_discursos_deputado(160575))

```

<httr2_request>

```

GET https://dadosabertos.camara.leg.br/api/v2/deputados/160575/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&itens=100
Body: empty

```

Da mesma forma que obtemos os dados dos deputados, vamos fazer requisições para obter discursos de cada deputado:

```
df_discursos <- df_deputados |>
  mutate(req_discursos = map(id, req_discursos_deputado)) |>      # montar a requisição chamada
  mutate(resp_discursos = req_perform_parallel(req_discursos)) |> # executar requisições em paralelo
  mutate(dados_discursos = map(resp_discursos, resp_body_json))    # ler dados das respostas
```

```
[working] (627 + 0) -> 10 -> 8 | >----- 1%
[working] (625 + 0) -> 10 -> 10 | >----- 2%
[working] (609 + 0) -> 10 -> 26 | =>----- 4%
[working] (582 + 0) -> 10 -> 53 | ==>----- 8%
[working] (565 + 0) -> 9 -> 71 | ===>----- 11%
[working] (524 + 0) -> 10 -> 111 | =====>----- 17%
[working] (499 + 0) -> 10 -> 136 | =====>----- 21%
[working] (463 + 0) -> 10 -> 172 | =====>----- 27%
[working] (434 + 0) -> 10 -> 201 | =====>----- 31%
[working] (405 + 0) -> 10 -> 230 | =====>----- 36%
[working] (376 + 0) -> 10 -> 259 | =====>----- 40%
[working] (353 + 0) -> 10 -> 282 | =====>----- 44%
[working] (346 + 0) -> 10 -> 289 | =====>----- 45%
[working] (326 + 0) -> 10 -> 309 | =====>----- 48%
[working] (293 + 0) -> 10 -> 342 | =====>----- 53%
```

```

[working] (269 + 0) -> 10 -> 366 | =====>----- 57%

[working] (238 + 0) -> 10 -> 397 | =====>----- 62%

[working] (219 + 0) -> 10 -> 416 | =====>----- 64%

[working] (190 + 0) -> 10 -> 445 | =====>----- 69%

[working] (157 + 0) -> 10 -> 478 | =====>----- 74%

[working] (124 + 0) -> 10 -> 511 | =====>----- 79%

[working] (100 + 0) -> 10 -> 535 | =====>----- 83%

[working] (71 + 0) -> 10 -> 564 | =====>----- 87%

[working] (48 + 0) -> 10 -> 587 | =====>----- 91%

[working] (17 + 0) -> 10 -> 618 | =====>- 96%

[working] (0 + 0) -> 5 -> 640 | =====> 99%

[working] (0 + 0) -> 0 -> 645 | =====> 100%

```

```
print(df_discurso$dados_discurso[[2]])
```

```

$dados
$dados[[1]]
$dados[[1]]$dataHoraInicio
[1] "2025-04-29T13:55"

$dados[[1]]$dataHoraFim
NULL

$dados[[1]]$uriEvento
[1] ""

$dados[[1]]$faseEvento

```

\$dados[[1]]\$faseEvento\$titulo
[1] "Encerramento"

\$dados[[1]]\$faseEvento\$dataHoraInicio
NULL

\$dados[[1]]\$faseEvento\$dataHoraFim
NULL

\$dados[[1]]\$tipoDiscurso
[1] "DISCURSO ENCAMINHADO"

\$dados[[1]]\$urlTexto
[1] "https://imagem.camara.gov.br/dc_20b.asp?largura=&altura=&tipoForm=diarios&selCodColecao"

\$dados[[1]]\$urlAudio
NULL

\$dados[[1]]\$urlVideo
NULL

\$dados[[1]]\$keywords
[1] "Deputado federal, Missão oficial, China, Balanço, Amapá, Fonte alternativa de energia, Desenvolvimento"

\$dados[[1]]\$sumario
[1] "O Deputado relatou sua participação em missão oficial à China, realizada entre 17 e 27 de março de 2025, com foco em temas de desenvolvimento econômico e energético."

\$dados[[1]]\$transcricao
[1] "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAVACHO (SEM REGISTRO TAQUIGRÁFICO)"

\$dados[[2]]
\$dados[[2]]\$dataHoraInicio
[1] "2025-04-02T10:32"

\$dados[[2]]\$dataHoraFim
NULL

\$dados[[2]]\$uriEvento
[1] ""

\$dados[[2]]\$faseEvento

\$dados[[2]]\$faseEvento\$titulo

[1] "Homenagem"

\$dados[[2]]\$faseEvento\$dataHoraInicio

NULL

\$dados[[2]]\$faseEvento\$dataHoraFim

NULL

\$dados[[2]]\$tipoDiscurso

[1] "HOMENAGEM"

\$dados[[2]]\$urlTexto

[1] "https://imagem.camara.gov.br/dc_20b.asp?largura=&altura=&tipoForm=diarios&selCodColecao"

\$dados[[2]]\$urlAudio

NULL

\$dados[[2]]\$urlVideo

NULL

\$dados[[2]]\$keywords

[1] "Dia Nacional de Conscientização sobre o Autismo,Homenagem,Sessão solene"

\$dados[[2]]\$sumario

[1] "O Deputado discursou na sessão solene em homenagem ao Dia Mundial de Conscientização sol"

\$dados[[2]]\$transcricao

[1] "O SR. ACÁCIO FAVACHO (Bloco/MDB - AP. Sem revisão do orador.) - Presidente, bom dia. Ob
o para Brasília para dar a ele um tratamento adequado. E, no dia de hoje, ela apresentou ess

\$dados[[3]]

\$dados[[3]]\$dataHoraInicio

[1] "2025-04-01T13:55"

\$dados[[3]]\$dataHoraFim

NULL

\$dados[[3]]\$uriEvento

[1] ""

```

$dados[[3]]$faseEvento
$dados[[3]]$faseEvento$titulo
[1] "Encerramento"

$dados[[3]]$faseEvento$dataHoraInicio
NULL

$dados[[3]]$faseEvento$dataHoraFim
NULL

$dados[[3]]$tipoDiscurso
[1] "DISCURSO ENCAMINHADO"

$dados[[3]]$urlTexto
[1] "https://imagem.camara.gov.br/dc_20b.asp?largura=&altura=&tipoForm=diarios&selCodColecao"

$dados[[3]]$urlAudio
NULL

$dados[[3]]$urlVideo
NULL

$dados[[3]]$keywords
[1] "Macapá (AP),Habitação,Política habitacional,Programa Minha Casa, Minha Vida (PMCMV),Min"

$dados[[3]]$sumario
[1] "O Deputado celebrou os avanços do Programa Casa Macapá, vinculado ao Programa Minha Casa"

$dados[[3]]$transcricao
[1] "DISCURSO NA ÍNTEGRA ENCAMINHADO PELO SR. DEPUTADO ACÁCIO FAVACHO (SEM REGISTRO TAQUIGRÁF"

$links
$links[[1]]
$links[[1]]$rel
[1] "self"

$links[[1]]$href
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/204379/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&itens=100"

```

```

$links[[2]]
$links[[2]]$rel
[1] "first"

$links[[2]]$href
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/204379/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&pagina=1&itens=100"

$links[[3]]
$links[[3]]$rel
[1] "last"

$links[[3]]$href
[1] "https://dadosabertos.camara.leg.br/api/v2/deputados/204379/discursos?dataInicio=2025-04-01&dataFim=2025-04-30&pagina=1&itens=100"

```

Por fim, vamos extrair dos dados em colunas:

```

df_discursos <- df_discursos |>
  unnest_wider(dados_discursos) |> # desaninhar resposta em colunas (dados e links)
  unnest(dados) |> # desaninhar dados de discursos em linha (um deputado por linha)
  unnest_wider(dados) |> # desaninhar dados em colunas
  unnest_wider(faseEvento, names_sep = "_") |> # desaninhar "faseEvento"
  select(
    # colunas anteriores
    colnames(df_deputados),
    # novas colunas
    hora_inicio = dataHoraInicio,
    fase_evento = faseEvento_titulo,
    tipo_discurso = tipoDiscurso,
    keywords,
    sumario,
    transcricao
  )
head(df_discursos)

```

A tibble: 6 x 22

id	nome	uf	partido	uri	nome_civil	sexo	cpf	data_nascimento
<int>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>


```

1 204379 Acácio Fava~ AP      MDB      http~ ACÁCIO DA~ M      7428~ 1983-09-28
2 204379 Acácio Fava~ AP      MDB      http~ ACÁCIO DA~ M      7428~ 1983-09-28
3 204379 Acácio Fava~ AP      MDB      http~ ACÁCIO DA~ M      7428~ 1983-09-28
4 220714 Adail Filho  AM      REPUB~ http~ ADAIL JOS~ M      7726~ 1992-02-16
5 221328 Adilson Bar~ SP      PL       http~ ADILSON B~ M      0558~ 1964-06-14
6 221328 Adilson Bar~ SP      PL       http~ ADILSON B~ M      0558~ 1964-06-14
# i 13 more variables: data_falecimento <chr>, uf_nascimento <chr>,
#   municipio_nascimento <chr>, escolaridade <chr>, situacao <chr>,
#   data_situacao <chr>, ultimo_partido <chr>, hora_inicio <chr>,
#   fase_evento <chr>, tipo_discurso <chr>, keywords <chr>, sumario <chr>,
#   transcricao <chr>

```

Por fim, podemos extrair estatísticas sobre os oradores e salvar os dados em uma planilha CSV para usos futuros.

```
print(str_c("Número total de discursos: ", nrow(df_discursos)))
```

```
[1] "Número total de discursos: 1849"
```

```
print(str_c("Número total de oradores: ", df_discursos |> select(nome) |> n_distinct()))
```

```
[1] "Número total de oradores: 317"
```

```
print(str_c("Número total de partidos: ", df_discursos |> select(partido) |> n_distinct()))
```

```
[1] "Número total de partidos: 21"
```

```
print(str_c("Número total de estados: ", df_discursos |> select(uf) |> n_distinct()))
```

```
[1] "Número total de estados: 27"
```

Principais oradores:

```
df_discursos |>
  group_by(nome) |> # agrupar por nome do orador
  count(sort = TRUE) # contar e ordenar

```

```
# A tibble: 317 x 2
# Groups:   nome [317]
  nome          n
  <chr>        <int>
1 Cabo Gilberto Silva 57
2 Luiz Lima          48
3 Hildo Rocha        39
4 Marcel van Hattem   39
5 Bibó Nunes         36
6 Erika Kokay        36
7 Coronel Assis      30
8 Gilson Marques     30
9 Chico Alencar      29
10 Otoni de Paula     28
# i 307 more rows
```

Principais partidos:

```
df_discursos |>
  group_by(partido) |> # agrupar por partido do orador
  count(sort = TRUE)  # contar e ordenar
```

```
# A tibble: 21 x 2
# Groups:   partido [21]
  partido      n
  <chr>    <int>
1 PL      455
2 PT      373
3 NOVO    136
4 PSOL    129
5 UNIÃO   117
6 MDB     105
7 PSD     94
8 REPUBLICANOS 92
9 PSB     82
10 PCdoB   57
# i 11 more rows
```

Para facilitar novos processamentos, podemos salvar os dados em formato CSV:

```
write_csv(df_discursos, "discursos.csv")
```

Unindo todo o processo:

```
library(tidyverse)
library(httr2)

obter_discursos_camara <- function(data_inicio, data_fim, id_legislatura = 57) {
  print("1/3 - Buscando lista de deputados")
  url_api <- "https://dadosabertos.camara.leg.br/"

  extrair_dados_resposta <- function(resp) {
    resposta <- resp_body_json(resp, simplifyDataFrame = TRUE)
    return(resposta$dados)
  }

  df_deputados_partido <- request(url_api) |>
    req_url_path("api", "v2", "deputados") |>
    req_url_query(
      idLegislatura = id_legislatura, # número da legislatura
      nome = "", # parte do nome
      siglaUF = "", # sigla UF eleito
      siglaPartido = "", # sigla do partido
      siglaSexo = "", # sexo (M ou F)
    ) |>
    req_perform() |> # executa a requisição
    extrair_dados_resposta() |> # extrai item "dados"
    select( # seleciona colunas relevantes
      id,
      nome,
      uf = siglaUf,
      partido = siglaPartido,
      uri
    )

  print("2/3 - Buscando detalhes de cada deputado")

  df_deputados <- df_deputados_partido |>
    mutate(req_detalhes = map(uri, request)) |> # montar requisições
    mutate(resp_detalhes = req_perform_parallel(req_detalhes)) |> # buscar detalhes
    mutate(detalhes = map(resp_detalhes, extrair_dados_resposta)) |> # extrair detalhes das 1
    unnest_wider(detalhes, names_sep = "_") |> # desaninhar detalhes
```

```

unnest_wider(detalhes_ultimoStatus, names_sep = "_") |> # desaninhar último status
select(                                              # selecionar colunas
  # colunas antigas
  colnames(df_deputados_partido),
  # novas colunas
  nome_civil = detalhes_nomeCivil,
  sexo = detalhes_sexo,
  data_nascimento = detalhes_dataNascimento,
  data_falecimento = detalhes_dataFalecimento,
  uf_nascimento = detalhes_ufNascimento,
  municipio_nascimento = detalhes_municipioNascimento,
  escolaridade = detalhes_escolaridade,
  situacao = detalhes_ultimoStatus_situacao,
  data_situacao = detalhes_ultimoStatus_data,
  ultimo_partido = detalhes_ultimoStatus_siglaPartido
) |>
filter(partido == ultimo_partido) |> # considerar apenas último partido
distinct()                          # filtrar dados duplicados

print("3/3 - Buscando discursos de cada deputado")

req_discursos_deputado <- function(id_deputado) {
  req <- request(url_api) |>
  req_url_path("api", "v2", "deputados", id_deputado, "discursos") |>
  req_url_query(
    dataInicio = data_inicio,
    dataFim = data_fim,
  )
  return(req)
}

resp_discursos_deputados <- df_deputados |>
mutate(req_discursos = map(id, req_discursos_deputado)) |>
mutate(resp_discursos = req_perform_parallel(req_discursos))

df_discursos <- resp_discursos_deputados |>
mutate(discursos = map(resp_discursos, extrair_dados_resposta)) |> # respostas
filter(lengths(discursos) > 0) |> # filtrar linhas sem discurso
unnest(discursos) |> # desaninhar dados de discursos
unnest_wider(faseEvento, names_sep = "_") |> # desaninhar dados de evento
select(                                              # selecionar colunas relevantes
  # colunas anteriores

```

```

        colnames(df_deputados),
        # novas colunas
        hora_inicio = dataHoraInicio,
        fase_evento = faseEvento_titulo,
        tipo_discurso = tipoDiscurso,
        keywords,
        sumario,
        transcricao
    )

    return(df_discursos)
}

```

```

dt_discursos_camara <- obter_discursos_camara(
    id_legislatura = 57,
    data_inicio = "2025-04-01",
    data_fim = "2025-04-30"
)

```

```

[1] "1/3 - Buscando lista de deputados"
[1] "2/3 - Buscando detalhes de cada deputado"

```

```

[working] (699 + 0) -> 8 -> 15 | =>----- 2%

```

```

[working] (670 + 0) -> 10 -> 42 | ==>----- 6%

```

```

[working] (628 + 0) -> 11 -> 83 | ===>----- 11%

```

```

Waiting 28s for rate limit =>-----

```

```

Waiting 28s for rate limit ====>-----

```

```

Waiting 28s for rate limit =====>-----

```

```

Waiting 28s for rate limit =====>-----

```

```

Waiting 28s for rate limit =====>-----

```

```

Waiting 28s for rate limit =====>-----

```

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>

Waiting 28s for rate limit =====>

[working] (628 + 0) -> 11 -> 83 | ===>----- 11%

[working] (576 + 1) -> 10 -> 136 | =====>----- 19%

Waiting 28s for rate limit =>-----

Waiting 28s for rate limit ===>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

[working] (576 + 1) -> 10 -> 136 | =====>----- 19%

[working] (522 + 2) -> 10 -> 190 | =====>----- 26%

[working] (478 + 2) -> 10 -> 234 | =====>----- 32%

Waiting 28s for rate limit =>-----

Waiting 28s for rate limit ==>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

Waiting 28s for rate limit =====>-----

[working] (478 + 2) -> 10 -> 234 | =====>----- 32%

[working] (422 + 3) -> 8 -> 292 | =====>----- 40%

[working] (379 + 3) -> 10 -> 333 | =====>----- 46%

[working] (338 + 3) -> 9 -> 375 | =====>----- 52%

[working] (294 + 3) -> 10 -> 418 | =====>----- 58%

```

[working] (250 + 3) -> 10 -> 462 | =====>----- 64%
[working] (205 + 3) -> 10 -> 507 | =====>----- 70%
[working] (160 + 3) -> 10 -> 552 | =====>----- 76%
[working] (115 + 3) -> 10 -> 597 | =====>----- 83%
[working] (70 + 3) -> 10 -> 642 | =====>----- 89%
[working] (27 + 3) -> 10 -> 685 | =====>----- 95%
[working] (0 + 3) -> 1 -> 721 | =====>----- 100%
[working] (0 + 3) -> 0 -> 722 | =====>----- 100%

```

[1] "3/3 - Buscando discursos de cada deputado"

```

[working] (616 + 0) -> 10 -> 19 | ==>----- 3%
[working] (583 + 0) -> 10 -> 52 | ==>----- 8%
[working] (550 + 0) -> 10 -> 85 | ====>----- 13%
[working] (518 + 0) -> 10 -> 117 | =====>----- 18%
[working] (487 + 0) -> 10 -> 148 | =====>----- 23%
[working] (459 + 0) -> 10 -> 176 | =====>----- 27%
[working] (427 + 0) -> 10 -> 208 | =====>----- 32%
[working] (393 + 0) -> 10 -> 242 | =====>----- 38%
[working] (378 + 0) -> 10 -> 257 | =====>----- 40%
[working] (339 + 0) -> 10 -> 296 | =====>----- 46%
[working] (308 + 0) -> 10 -> 327 | =====>----- 51%
[working] (282 + 0) -> 10 -> 353 | =====>----- 55%
[working] (246 + 0) -> 10 -> 389 | =====>----- 60%
[working] (217 + 0) -> 10 -> 418 | =====>----- 65%
[working] (172 + 0) -> 11 -> 462 | =====>----- 72%
[working] (132 + 0) -> 10 -> 503 | =====>----- 78%
[working] (85 + 0) -> 10 -> 550 | =====>----- 85%
[working] (45 + 0) -> 10 -> 590 | =====>----- 91%
[working] (8 + 0) -> 10 -> 627 | =====>----- 97%
[working] (0 + 0) -> 0 -> 645 | =====>----- 100%

```

```
head(dt_discursos_camara)
```

```
# A tibble: 6 x 21
```

	id	nome	uf	partido	uri	nome_civil	sexo	data_nascimento
	<int>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>
1	204379	Acácio Favacho	AP	MDB	http~	ACÁCIO DA~	M	1983-09-28
2	204379	Acácio Favacho	AP	MDB	http~	ACÁCIO DA~	M	1983-09-28
3	204379	Acácio Favacho	AP	MDB	http~	ACÁCIO DA~	M	1983-09-28
4	220714	Adail Filho	AM	REPUBLICA~	http~	ADAIL JOS~	M	1992-02-16

```

5 221328 Adilson Barroso SP      PL      http~ ADILSON B~ M      1964-06-14
6 221328 Adilson Barroso SP      PL      http~ ADILSON B~ M      1964-06-14
# i 13 more variables: data_falecimento <chr>, uf_nascimento <chr>,
#   municipio_nascimento <chr>, escolaridade <chr>, situacao <chr>,
#   data_situacao <chr>, ultimo_partido <chr>, hora_inicio <chr>,
#   fase_evento <chr>, tipo_discurso <chr>, keywords <chr>, sumario <chr>,
#   transcricao <chr>

```

```
write_csv(dt_discursos_camara, "discursos_camara_abril_2025.csv")
```

A.2 Coletando dados da API do Senado

De maneira análoga, podemos definir uma função para obter dados de discursos do senado:

```

obter_discursos_senado <- function(data_inicio, data_fim) {
  url_discursos <- "https://legis.senado.leg.br/dadosabertos/plenario/lista/discursos/"
  req_discursos <- request(url_discursos) |>
    req_url_path_append(data_inicio, data_fim) |>
    req_headers("Accept" = "application/json")

  resp_discursos <- req_perform(req_discursos)

  print("1/3 - Obtendo dados de discursos")

  dt_discursos <- resp_discursos |>
    resp_body_json() |>                                     # extrair dados da resposta em JSON
    pluck("DiscursosSessao", "Sessoes") |>                 # acessar dados dentro de DiscursosSessao
    as_tibble() |>                                           # transformar em tabela
    unnest_wider(Sessao) |>                                  # desaninhar dados das sessões
    unnest(Pronunciamentos) |>                               # desaninhar pronunciamentos
    unnest_longer(Pronunciamentos) |>                       # desaninhar pronunciamentos da sessão
    unnest_wider(Pronunciamentos) |>                       # desaninhar dados do pronunciamento
    unnest_wider(TipoUsoPalavra, names_sep = '_') |>        # desaninhar tipo de uso da palavra
    filter(!is.na(CodigoParlamentar)) |>                   # filtrar discursos de não-parlamentares
    select(                                                  # selecionar colunas
      id,
      codigo_parlamentar = CodigoParlamentar,
      nome = NomeAutor,
      partido = Partido,
      uf = UF,
      tipo_sessao = DescricaoSessao,

```



```

    tipo_uso_palavra = TipoUsoPalavra_Descricao,
    data = Data,
    sumario = Resumo,
    keywords = Indexacao,
    transcricao = TextoIntegralTxt
  )

print("2/3 - Obtendo inteiro teor")

dt_discursos_inteiro <- dt_discursos |>
  mutate(transcricao = map(transcricao, request)) |> # requisição da transcrição
  mutate(transcricao = req_perform_parallel(transcricao)) |> # buscar transcrições
  mutate(transcricao = map(transcricao, resp_body_string)) # ler transcrições das respostas

req_detalhes_senador <- function(codigo_parlamentar) {
  url_senadores <- "https://legis.senado.leg.br/dadosabertos/senador/"
  req <- request(url_senadores) |>
    req_url_path_append(codigo_parlamentar) |>
    req_headers(Accept = "application/json")
  return(req)
}

print("3/3 - Obtendo detalhes dos senadores")

dt_detalhes_senadores <- dt_discursos_inteiro |>
  select(codigo_parlamentar) |>
  distinct() |>
  mutate(req_senador = map(codigo_parlamentar, req_detalhes_senador)) |>
  mutate(resp_senador = req_perform_parallel(req_senador)) |>
  mutate(detalhes = map(resp_senador, resp_body_json)) |>
  unnest(detalhes) |> # desaninhar detalhes
  unnest_wider(detalhes) |> # desaninhar dados de detalhes
  unnest_wider(Parlamentar) |> # desaninhar dados do parlamentar
  unnest_wider(DadosBasicosParlamentar) |> # desaninhar dados básicos
  unnest_wider(IdentificacaoParlamentar) |> # desaninhar identificação
  select( # selecionar colunas
    codigo_parlamentar,
    nome_civil = NomeCompletoParlamentar,
    sexo = SexoParlamentar,
    data_nascimento = DataNascimento,
    uf_nascimento = UfNaturalidade,
    municipio_nascimento = Naturalidade
  )

```

```

    )

    dt_discursos_senado <- dt_discursos_inteiro |>
      inner_join(dt_detalhes_senadores, by = join_by(codigo_parlamentar))

    return(dt_discursos_senado)
  }

```

```

df_discursos_senado <- obter_discursos_senado(
  data_inicio = "20250401",
  data_fim = "20250430"
)

```

```

[1] "1/3 - Obtendo dados de discursos"
[1] "2/3 - Obtendo inteiro teor"

```

```

Waiting 15s for rate limit ==>-----
Waiting 15s for rate limit ==>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>----

```

```

Waiting 15s for rate limit =====>

```

```

[working] (153 + 8) -> 10 -> 188 | =====>----- 54%

```

```

Waiting 15s for rate limit ==>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>-----

```

```

Waiting 15s for rate limit =====>-----

```

Waiting 15s for rate limit =====>----

Waiting 15s for rate limit =====>

```
[working] (153 + 8) -> 10 -> 188 | =====>----- 54%
[working] (21 + 11) -> 10 -> 320 | =====>---- 91%
[working] (0 + 11) -> 0 -> 351 | =====> 100%
```

[1] "3/3 - Obtendo detalhes dos senadores"

Waiting 15s for rate limit ==>-----

Waiting 15s for rate limit =====>-----

Waiting 15s for rate limit =====>-----

Waiting 15s for rate limit =====>-----

Waiting 15s for rate limit =====>----

Waiting 15s for rate limit =====>

```
head(df_discursos_senado)
```

A tibble: 6 x 16

	id	codigo_parlamentar	nome	partido	uf	tipo_sessao	tipo_uso_palavra
	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>
1	513287	5976	Eduardo ~	NOVO	CE	Sessão Del~	Discurso
2	513284	1173	Wellingt~	PL	MT	Sessão Del~	Pela Liderança
3	513283	6304	Margaret~	PSD	MT	Sessão Del~	Discurso
4	513280	4560	Sérgio P~	PSD	AC	Sessão Del~	Discurso
5	513279	4531	Jayme Ca~	UNIÃO	MT	Sessão Del~	Pela Liderança
6	513278	5906	Dra. Eud~	PL	AL	Sessão Del~	Discurso

i 9 more variables: data <chr>, sumario <chr>, keywords <chr>,
transcricao <list>, nome_civil <chr>, sexo <chr>, data_nascimento <chr>,
uf_nascimento <chr>, municipio_nascimento <chr>

```
write_csv(df_discursos_senado, "discursos_senado_abril_2025.csv")
```